

AutoDev

// especificación_de_requisitos.md

document_summary

Este documento define las especificaciones técnicas y funcionales de AutoDev, una plataforma comunitaria para desarrolladores enfocada en automatización, inteligencia artificial y desarrollo web. Cubre el ciclo completo: requisitos funcionales, casos de uso, criterios de aceptación, escenarios de prueba BDD automatizables, arquitectura del sistema basada en Firebase (Auth, Firestore, Cloud Functions, Storage, FCM y Analytics) y diseño detallado de la base de datos NoSQL con reglas de seguridad y triggers serverless.

// VERSION

1.0 — Release Candidate

// FECHA

10 de Julio, 2026

// AUTOR

Equipo de Producto — AutoDev

// METODOLOGÍA

SDD + BDD (Gherkin)

auth

foro

cursos

recursos

directos

miembros

ranking

gamificacion

firestore

cloud_functions

Tabla de Contenidos

Capítulo 1 — Introducción	6
1.1 Propósito del documento	6
1.2 Alcance del sistema	6
1.3 Definiciones, acrónimos y glosario	6
1.4 Referencias	7
1.5 Visión general del documento	7
Capítulo 2 — Descripción General	9
2.1 Perspectiva del producto	9
2.2 Funciones del producto	9
2.3 Características de los usuarios	10
2.4 Restricciones	10
2.5 Suposiciones y dependencias	10
Capítulo 3 — Requisitos Funcionales	12
3.1 Módulo de Autenticación (RF-001 a RF-008)	12
3.2 Módulo de Onboarding y Perfil (RF-009 a RF-018)	12
3.3 Módulo de Foro (RF-019 a RF-028)	13
3.4 Módulo de Cursos (RF-029 a RF-035)	13
3.5 Módulo de Recursos (RF-036 a RF-043)	14
3.6 Módulo de Directos (RF-044 a RF-048)	14
3.7 Módulo de Miembros (RF-049 a RF-053)	15
3.8 Módulo de Ranking (RF-054 a RF-057)	15
3.9 Módulo de Gamificación (RF-058 a RF-062)	15

3.10 Módulo de Notificaciones (RF-063 a RF-067)	16
3.11 Módulo de Administración (RF-068 a RF-075)	16

Capítulo 4 — Requisitos No Funcionales **17**

4.1 Rendimiento	17
4.2 Escalabilidad	17
4.3 Seguridad	18
4.4 Disponibilidad y observabilidad	18
4.5 Usabilidad y accesibilidad	19
4.6 Mantenibilidad y compatibilidad	19

Capítulo 5 — Casos de Uso Detallados **21**

5.1 Diagrama de casos de uso (descripción textual)	21
5.2 Plantilla de caso de uso	21
5.3 CU-001 — Registro de cuenta	21
5.4 CU-002 — Onboarding de perfil	22
5.5 CU-003 — Publicar post en foro	23
5.6 CU-004 — Comentar post	24
5.7 CU-005 — Filtrar miembros por especialidad	24
5.8 CU-006 — Publicar recurso	25
5.9 CU-007 — Inscribirse a directo	25
5.10 CU-008 — Visualizar ranking semanal	26
5.11 CU-009 — Completar misión	26
5.12 CU-010 — Recibir notificación push	26
5.13 CU-011 — Moderar contenido (admin/mod)	27
5.14 CU-012 — Buscar recurso por tipo	27

Capítulo 6 — Criterios de Aceptación **28**

6.1 Criterios de aceptación por módulo	28
Capítulo 7 — Escenarios de Prueba Automatizables (BDD)	31
7.1 Feature: Autenticación	31
7.2 Feature: Onboarding	32
7.3 Feature: Foro	33
7.4 Feature: Recursos	34
7.5 Feature: Directos	35
7.6 Feature: Ranking y Gamificación	35
7.7 Feature: Notificaciones	36
7.8 Feature: Administración	37
Capítulo 8 — Estrategia de Pruebas	38
8.1 Pirámide de pruebas	38
8.2 Stack de pruebas detallado	38
8.3 Cobertura mínima	39
8.4 CI/CD Pipeline	39
8.5 Datos de prueba y seed	39
8.6 Gestión de ambientes	39
Capítulo 9 — Arquitectura del Sistema	40
9.1 Vista general	40
9.2 Componentes y responsabilidades	40
9.3 Patrones arquitectónicos	41
9.3.1 Repository Pattern en el frontend	41
9.3.2 Event-driven backend con triggers de Firestore	41
9.3.3 Serverless con escalado a cero	41

9.3.4 Single source of truth en Firestore Rules	41
9.4 Flujos de datos principales	41
9.4.1 Flujo: Publicación de un post	41
9.4.2 Flujo: Cálculo del ranking semanal	41
9.4.3 Flujo: Notificación push tras comentario	41
9.5 Seguridad perimetral	42
9.6 Estrategia multi-región y escalado	42
Capítulo 10 — Diseño de Base de Datos (Firestore)	43
10.1 Modelo de datos general	43
10.2 Colección: users	43
10.3 Colección: posts y subcolección comments	43
10.4 Colección: resources	44
10.5 Colección: courses y subcolección lessons	44
10.6 Colección: lives y subcolecciones	45
10.7 Colección: rankings (histórico)	46
10.8 Colecciones: missions, achievements, notifications, audit_logs	46
10.9 Índices compuestos necesarios	47
10.10 Reglas de seguridad Firestore	48
10.11 Triggers de Cloud Functions	49
10.12 Snippet de Cloud Function: onPostCreate	50
Capítulo 11 — Roadmap y Entregables	52
11.1 Fases del roadmap	52
11.2 Criterios de salida por fase	52
11.3 Riesgos y mitigaciones	52
Capítulo 12 — Apéndices	54

Apendice A — Glosario completo	54
Apendice B — Matriz de trazabilidad	54
Apendice C — Convenciones de nombrado	56
Apendice D — Documentos relacionados	57

Capítulo 1 — Introducción

1.1 Propósito del documento

El presente documento constituye la Especificación de Requisitos de Software (SRS) de **AutoDev**, elaborada bajo la metodología *Specification Driven Development* (SDD). Su objetivo principal es establecer una base formal y verificable que articule, en un único artefacto, los requisitos funcionales, los casos de uso, los criterios de aceptación, los escenarios de prueba automatizables y el diseño arquitectónico del sistema. Esta especificación sirve como contrato entre los stakeholders de producto, el equipo de ingeniería y los validadores de calidad, garantizando que toda decisión técnica posterior pueda trazarse a un requisito explícito y a un escenario de aceptación ejecutable.

La elección del enfoque SDD no es accidental: AutoDev es una plataforma comunitaria orientada a desarrolladores que valoran la precisión técnica y la reproducibilidad. Un documento de especificación tradicional, ambiguo o descriptivo, generaría interpretaciones divergentes entre frontend, backend y QA. Al adoptar SDD junto con Behavior Driven Development (BDD) para los escenarios de aceptación, aseguramos que cada funcionalidad pueda validarse mediante pruebas automatizadas escritas en lenguaje Gherkin, ejecutables contra el Firebase Emulator Suite y desplegadas en un pipeline de integración continua.

Este documento está dirigido a product owners, ingenieros fullstack, ingenieros de QA, arquitectos de soluciones y a cualquier revisor técnico que necesite comprender el alcance del sistema antes de su implementación. La versión 1.0 corresponde a la línea base inicial sobre la cual se construirán las iteraciones incrementales posteriores.

1.2 Alcance del sistema

AutoDev es una plataforma web responsive orientada a la comunidad de desarrolladores interesados en automatización, inteligencia artificial y desarrollo web moderno. El sistema permite a los usuarios registrarse, configurar su perfil técnico, publicar y consumir contenido en un foro comunitario, acceder a cursos y recursos educativos, participar en sesiones en vivo, visualizar el directorio de miembros y competir en un ranking semanal impulsado por un sistema de gamificación basado en puntos de experiencia (XP).

El alcance funcional cubierto por esta especificación incluye: (i) autenticación de usuarios mediante Firebase Authentication con email y contraseña; (ii) onboarding guiado en cuatro pasos (registro, configuración de perfil, selección de intereses y nivel, presentación comunitaria); (iii) foro público con posts, comentarios, reacciones y filtros por etiqueta; (iv) catálogo de cursos con seguimiento de progreso por lección; (v) repositorio de recursos tipificados (Skill, Plugin, Subagent, MCP Server, Agent Team, Tutorial); (vi) calendario de sesiones en vivo con reserva de cupos limitados; (vii) directorio de miembros con filtros por nivel y especialidad; (viii) leaderboard semanal con cálculo de XP y reparto de premios; (ix) sistema de misiones y logros; (x) notificaciones in-app, por email y push; (xi) panel de administración con moderación de contenido y métricas globales.

Quedan fuera del alcance de esta versión: integraciones de pago con Stripe (planificada para la fase 4), mensajería directa entre miembros (fase 5), aplicación móvil nativa (la plataforma web es responsive pero no se empaqueta como app instalable en esta versión), integraciones con GitHub/GitLab para sincronización de repositorios, y funcionalidades de IA generativa para recomendación de contenido.

1.3 Definiciones, acrónimos y glosario

Para asegurar la consistencia terminológica a lo largo del documento, se establecen las siguientes definiciones. Cualquier término técnico no listado aquí conserva su significado estándar en la literatura de ingeniería de software.

Término	Definición
SDD	Specification Driven Development. Metodología que prioriza la especificación formal de requisitos antes de la implementación.
SRS	Software Requirements Specification. Documento que describe requisitos funcionales y no funcionales del sistema.
BDD	Behavior Driven Development. Práctica que expresa requisitos como comportamientos en lenguaje natural estructurado (Gherkin).
Gherkin	Lenguaje estructurado (Given/When/Then) para describir escenarios de prueba ejecutables.
Firebase	Plataforma Backend-as-a-Service de Google Cloud que provee Auth, Firestore, Functions, Storage, Hosting y Analytics.
Firestore	Base de datos NoSQL orientada a documentos, en tiempo real, con sincronización offline.
Cloud Functions	Funciones serverless de Firebase ejecutadas en respuesta a eventos (HTTP, triggers de Firestore, scheduled).
FCM	Firebase Cloud Messaging. Servicio de notificaciones push multiplataforma.
XP	Experience Points. Unidad de puntaje en el sistema de gamificación de AutoDev.
Onboarding	Proceso guiado de registro y configuración inicial de un nuevo usuario.
Leaderboard	Tabla de clasificación semanal de usuarios ordenada por XP acumulada.
MCP Server	Model Context Protocol Server. Recurso tipificado en AutoDev para integraciones con LLMs.
Misión	Conjunto de tareas asignadas al usuario que otorgan XP al completarse.
App Check	Servicio Firebase que protege backends contra abuso verificando la identidad de la app cliente.

1.4 Referencias

Este documento se alinea con las siguientes normas y referencias técnicas: IEEE Std 830-1998 (Recommended Practice for Software Requirements Specifications), la documentación oficial de Firebase Authentication, Firestore Data Model, Cloud Functions v2 y Firebase Security Rules (versión actualizada a julio 2026), la guía WCAG 2.1 del W3C para accesibilidad web, y la especificación de Gherkin mantenida por Cucumber. Las decisiones arquitectónicas se documentan siguiendo el patrón ADR (Architecture Decision Records) y los principios SOLID para el diseño de componentes.

1.5 Visión general del documento

El documento se organiza en doce capítulos. El Capítulo 2 ofrece una descripción general del producto, los roles de usuario y las restricciones. El Capítulo 3 lista los requisitos funcionales (RF-001 a RF-060) agrupados por módulo. El Capítulo 4 especifica los requisitos no funcionales con métricas verificables. El Capítulo 5 detalla doce casos de

uso clave con flujos principal y alternativos. El Capítulo 6 presenta la matriz de criterios de aceptación. El Capítulo 7 contiene los escenarios BDD en Gherkin para los flujos automatizables. El Capítulo 8 describe la estrategia de pruebas y la pirámide adoptada. El Capítulo 9 define la arquitectura del sistema sobre Firebase. El Capítulo 10 detalla el modelo de datos en Firestore, índices, reglas de seguridad y triggers de Cloud Functions. El Capítulo 11 presenta el roadmap en cuatro fases y los riesgos identificados. El Capítulo 12 incluye los apéndices: glosario, matriz de trazabilidad y convenciones.

Capítulo 2 — Descripción General

2.1 Perspectiva del producto

AutoDev es un producto independiente que se posiciona como una comunidad técnica especializada para desarrolladores hispanohablantes interesados en automatización, inteligencia artificial aplicada y desarrollo web moderno. A diferencia de plataformas genéricas como Stack Overflow o Reddit, AutoDev combina tres dimensiones que normalmente viven separadas: el aprendizaje estructurado (cursos y tutoriales), la colaboración comunitaria (foro y directorio de miembros) y la motivación sostenida (gamificación con XP, misiones y ranking semanal). El sistema no es un componente embebido dentro de un producto mayor; es una plataforma autónoma que se integra con servicios externos únicamente para autenticación (Firebase Auth), almacenamiento (Firestore y Storage) y notificaciones push (FCM).

La interfaz está diseñada con una estética de terminal moderna: tema oscuro, tipografía monoespaciada para elementos decorativos, prefijos de shell (\$) y rutas estilo Unix (*autodev ~/foro*). Esta elección visual no es decorativa sino funcional: comunica pertenencia a una audiencia técnica y reduce la fricción estética para desarrolladores que pasan largas jornadas en editores de código con temas similares. El frontend es responsive y se optimiza para resoluciones de escritorio (1280px y superiores) primero, con adaptaciones para tablet (768px) y móvil (375px).

2.2 Funciones del producto

El producto se organiza en once módulos funcionales. Cada módulo agrupa un conjunto cohesionado de requisitos funcionales que se detallan en el Capítulo 3. La tabla siguiente resume los módulos, su propósito y el identificador de los requisitos asociados.

Módulo	Propósito	RF asociados
Autenticación	Registro, login, recuperación de contraseña y verificación de email	RF-001 a RF-008
Onboarding y Perfil	Configuración inicial del perfil, intereses, nivel y presentación	RF-009 a RF-018
Foro	Publicación, comentario, reacción, filtrado y búsqueda de posts	RF-019 a RF-028
Cursos	Catálogo de cursos y seguimiento de progreso por lección	RF-029 a RF-035
Recursos	Publicación y consumo de recursos tipificados (Skill, Plugin, etc.)	RF-036 a RF-043
Directos	Calendario de sesiones en vivo y reserva de cupos	RF-044 a RF-048
Miembros	Directorio con filtros y perfil público	RF-049 a RF-053
Ranking	Leaderboard semanal con cálculo de XP y premios	RF-054 a RF-057
Gamificación	Misiones, logros, niveles y asignación de XP	RF-058 a RF-062
Notificaciones	Notificaciones in-app, email y push FCM	RF-063 a RF-067
Administración	Moderación, gestión de usuarios y métricas globales	RF-068 a RF-075

2.3 Características de los usuarios

AutoDev define cinco roles de usuario con permisos diferenciados. El diseño de permisos sigue el principio de mínimo privilegio: cada rol recibe solo los permisos necesarios para sus objetivos. Las reglas de seguridad de Firestore (Capítulo 10) hacen cumplir estos permisos a nivel de base de datos, de modo que cualquier acceso no autorizado sea rechazado incluso si el frontend tuviera un bug.

Rol	Descripción	Permisos clave
Invitado	Usuario no autenticado que explora la plataforma.	Ver landing, ver recursos públicos, crear cuenta. No puede publicar ni interactuar.
Miembro	Usuario registrado con onboarding completo.	Publicar posts, comentar, reaccionar, reservar cupos en directos, ganar XP, participar en ranking.
Autor	Miembro con permisos de publicación de recursos y cursos.	Todos los permisos de Miembro más: publicar recursos, publicar cursos, ver métricas de su contenido.
Moderador	Miembro con permisos de moderación comunitaria.	Todos los permisos de Miembro más: ocultar posts, suspender usuarios temporalmente, marcar contenido inapropiado.
Administrador	Acceso total al panel de administración.	Todos los permisos más: gestionar usuarios y roles, gestionar cursos y recursos de cualquier autor, ver métricas globales, configurar plataforma.

2.4 Restricciones

El sistema está sujeto a las siguientes restricciones impuestas por el stack tecnológico y por decisiones de producto. Estas restricciones son vinculantes y cualquier propuesta de cambio debe someterse a un ADR (Architecture Decision Record) antes de su aprobación.

- El backend debe ejecutarse exclusivamente sobre Firebase (Auth, Firestore, Cloud Functions, Storage, Hosting, FCM, Analytics). No se permiten servidores tradicionales provisioningados ni bases de datos SQL.
- El frontend debe construirse con Next.js 16 y React 19, desplegado en Firebase Hosting. El renderizado debe ser client-side para contenido dinámico y static generation para landing y páginas públicas.
- El idioma por defecto de la interfaz y del contenido generado por el sistema es español. La arquitectura debe estar preparada para i18n pero el lanzamiento v1.0 es monoidioma.
- Todas las Cloud Functions deben estar escritas en TypeScript con tipado estricto y usar el SDK v2 de Firebase Functions.
- Las reglas de seguridad de Firestore deben ser la única fuente de verdad para la autorización. El frontend no debe confiar en su propia lógica de permisos para ocultar acciones que el backend permitiría.
- El diseño debe ser responsive mobile-first con soporte mínimo para viewport de 375px. WCAG 2.1 nivel AA es obligatorio.

2.5 Suposiciones y dependencias

La especificación asume las siguientes suposiciones: (i) los usuarios finales disponen de un navegador moderno (Chrome, Firefox, Safari o Edge en sus últimas dos versiones) con JavaScript habilitado; (ii) Firebase se encuentra disponible en la región us-central1 con un SLA del 99.9% para Firestore y Cloud Functions; (iii) el servicio FCM opera sin restricciones en las regiones objetivo; (iv) los usuarios aceptan recibir notificaciones por email transaccional mediante el servicio configurado en Firebase (SendGrid o equivalente).

Las dependencias externas críticas son: Firebase Platform (sin alternativa de contingencia en esta versión), Google Cloud Storage para alojamiento de imágenes de recursos y avatares, y el proveedor de email transaccional configurado. Cualquier interrupción de estos servicios afecta la disponibilidad de funcionalidades específicas según se documenta en el Capítulo 4 (Requisitos No Funcionales).

Capítulo 3 — Requisitos Funcionales

Este capítulo lista los requisitos funcionales del sistema AutoDev. Cada requisito recibe un identificador único (RF-NNN), una descripción en una sola oración verificable, una prioridad (Alta, Media, Baja) y el módulo al que pertenece. La prioridad Alta corresponde a requisitos sin los cuales el MVP no puede lanzarse; Media corresponde a requisitos necesarios para la fase 2; Baja corresponde a mejoras diferibles. La trazabilidad con casos de uso, criterios de aceptación y escenarios BDD se presenta en el Apéndice B.

3.1 Módulo de Autenticación (RF-001 a RF-008)

ID	Descripción	Prioridad
RF-001	El sistema debe permitir a un usuario invitado registrarse proporcionando email y contraseña con mínimo 8 caracteres.	Alta
RF-002	El sistema debe verificar unicidad de email al registrar y rechazar registros con email ya existente.	Alta
RF-003	El sistema debe enviar un email de verificación tras el registro exitoso.	Media
RF-004	El sistema debe permitir a un usuario iniciar sesión con email y contraseña.	Alta
RF-005	El sistema debe permitir a un usuario cerrar sesión.	Alta
RF-006	El sistema debe permitir recuperar contraseña mediante enlace enviado al email registrado.	Alta
RF-007	El sistema debe bloquear temporalmente tras 5 intentos fallidos de login consecutivos durante 15 minutos.	Media
RF-008	El sistema debe persistir la sesión del usuario mediante token JWT con expiración de 30 días.	Alta

3.2 Módulo de Onboarding y Perfil (RF-009 a RF-018)

ID	Descripción	Prioridad
RF-009	Tras el registro, el sistema debe redirigir al usuario al wizard de onboarding de 4 pasos.	Alta
RF-010	El paso 1 del onboarding debe capturar el nombre para mostrar del usuario (mínimo 2 caracteres).	Alta
RF-011	El paso 2 del onboarding debe permitir seleccionar al menos un interés entre: Automatización, IA, Desarrollo Web, Comunidad.	Alta
RF-012	El paso 3 del onboarding debe permitir seleccionar exactamente un nivel de experiencia: Principiante, Intermedio, Avanzado.	Alta
RF-013	El paso 4 del onboarding debe capturar una presentación personal de mínimo 20 caracteres.	Alta
RF-014	El sistema debe impedir el acceso a las demás secciones hasta completar el onboarding.	Alta
RF-015	El sistema debe permitir al usuario editar su perfil (nombre, intereses, nivel, presentación) desde la página de perfil.	Alta

ID	Descripción	Prioridad
RF-016	El sistema debe permitir al usuario subir un avatar (JPG, PNG o WebP, máximo 2MB) almacenado en Cloud Storage.	Media
RF-017	El sistema debe generar un avatar por defecto con las iniciales del usuario si no se sube uno.	Media
RF-018	El sistema debe mostrar el nivel, XP acumulada y misiones completadas en el perfil público del usuario.	Media

3.3 Módulo de Foro (RF-019 a RF-028)

ID	Descripción	Prioridad
RF-019	El sistema debe permitir a un Miembro crear un post con título (máximo 100 caracteres), contenido (máximo 5000 caracteres) y hasta 3 etiquetas.	Alta
RF-020	El sistema debe permitir al autor editar su propio post dentro de los 30 minutos posteriores a la publicación.	Alta
RF-021	El sistema debe permitir al autor eliminar su propio post en cualquier momento.	Alta
RF-022	El sistema debe permitir a un Miembro comentar en cualquier post con texto de máximo 1000 caracteres.	Alta
RF-023	El sistema debe permitir a un Miembro reaccionar (like) a un post o comentario, con toggle (un like por usuario).	Alta
RF-024	El sistema debe listar los posts en orden cronológico descendente con paginación de 20 posts por página.	Alta
RF-025	El sistema debe permitir filtrar posts por etiqueta (automatización, IA, webapps, general) y combinar con búsqueda de texto.	Alta
RF-026	El sistema debe mostrar en cada post el avatar, nombre y nivel del autor, fecha relativa y conteo de comentarios/likes.	Alta
RF-027	El sistema debe notificar al autor de un post cuando reciba un nuevo comentario o like.	Media
RF-028	El sistema debe impedir la publicación de posts con contenido vacío o que incumpla el filtro de palabras prohibidas.	Media

3.4 Módulo de Cursos (RF-029 a RF-035)

ID	Descripción	Prioridad
RF-029	El sistema debe listar los cursos disponibles con título, autor, duración estimada y número de lecciones.	Alta
RF-030	El sistema debe permitir a un Miembro inscribirse en un curso (gratis).	Alta
RF-031	El sistema debe permitir a un Miembro marcar una lección como completada.	Alta

ID	Descripción	Prioridad
RF-032	El sistema debe mostrar el progreso del usuario por curso como porcentaje de lecciones completadas.	Alta
RF-033	El sistema debe otorgar XP al completar una lección (configurable por curso, default 50 XP).	Media
RF-034	El sistema debe permitir a un Autor crear cursos compuestos por lecciones con título, contenido y recursos adjuntos.	Media
RF-035	El sistema debe mostrar un certificado simulado (no válido oficialmente) al completar el 100% de un curso.	Baja

3.5 Módulo de Recursos (RF-036 a RF-043)

ID	Descripción	Prioridad
RF-036	El sistema debe permitir a un Autor publicar un recurso con título, descripción, tipo, nivel e imagen destacada.	Alta
RF-037	El sistema debe soportar los siguientes tipos de recurso: Skill, Plugin, Subagent, MCP Server, Agent Team, Tutorial.	Alta
RF-038	El sistema debe permitir filtrar recursos por tipo, nivel (Principiante, Intermedio, Avanzado) y búsqueda de texto.	Alta
RF-039	El sistema debe mostrar cada recurso con imagen destacada, título, descripción, etiqueta de nivel y botón de acción (./ver).	Alta
RF-040	El sistema debe permitir a un Miembro descargar o enlazar al recurso externo (URL o archivo en Storage).	Alta
RF-041	El sistema debe contar el número de descargas/visitas por recurso para mostrar popularidad.	Media
RF-042	El sistema debe permitir al Autor editar o eliminar sus propios recursos.	Alta
RF-043	El sistema debe permitir a un Miembro marcar un recurso como favorito y ver su lista de favoritos.	Media

3.6 Módulo de Directos (RF-044 a RF-048)

ID	Descripción	Prioridad
RF-044	El sistema debe mostrar un calendario de próximas sesiones en vivo con título, fecha, hora y cupo máximo.	Alta
RF-045	El sistema debe permitir a un Miembro reservar un cupo en una sesión en vivo gratuita.	Alta
RF-046	El sistema debe impedir reservas cuando el cupo esté agotado, mostrando lista de espera.	Alta
RF-047	El sistema debe enviar notificación push y email recordatorio 1 hora antes de la sesión.	Media
RF-048	El sistema debe permitir al Autor o Admin cancelar una sesión, liberando los cupos y notificando a los inscritos.	Media

3.7 Módulo de Miembros (RF-049 a RF-053)

ID	Descripción	Prioridad
RF-049	El sistema debe mostrar un directorio de miembros con avatar, nombre, nivel, fecha de unión, especializaciones, última actividad y métricas (posts, comentarios).	Alta
RF-050	El sistema debe permitir filtrar miembros por nivel (Principiante, Intermedio, Avanzado) y especialización.	Alta
RF-051	El sistema debe permitir ordenar miembros por recientes, más activos o todos.	Alta
RF-052	El sistema debe mostrar estadísticas globales: total de miembros, miembros activos hoy.	Media
RF-053	El sistema debe permitir ver el perfil público de un miembro con su actividad reciente (últimos 10 posts/comentarios).	Media

3.8 Módulo de Ranking (RF-054 a RF-057)

ID	Descripción	Prioridad
RF-054	El sistema debe calcular semanalmente un leaderboard con los 100 miembros con mayor XP acumulada en la semana.	Alta
RF-055	El sistema debe mostrar visualmente el top 3 con medallas (oro, plata, bronce) y posiciones 4-100 en tabla.	Alta
RF-056	El sistema debe reiniciar el leaderboard cada lunes a las 00:00 UTC-4 y archivar el ranking de la semana anterior.	Alta
RF-057	El sistema debe otorgar premios XP bonus a los top 3 (500, 300, 200 XP respectivamente) el día del reseteo.	Media

3.9 Módulo de Gamificación (RF-058 a RF-062)

ID	Descripción	Prioridad
RF-058	El sistema debe asignar XP por acciones: 10 XP por comentario, 25 XP por post, 50 XP por lección completada, 5 XP por like recibido.	Alta
RF-059	El sistema debe definir misiones como conjunto de tareas (ej. "Completa tu perfil" requiere avatar + presentación + 1 interés).	Alta
RF-060	El sistema debe otorgar XP bonus al completar una misión (configurable, default 100 XP).	Alta
RF-061	El sistema debe definir logros desbloqueables por hitos (ej. "Primer post", "10 comentarios", "Top 10 semanal").	Media
RF-062	El sistema debe mostrar un panel de progreso con misiones activas, logros obtenidos y XP actual/necesaria para subir de nivel.	Alta

3.10 Módulo de Notificaciones (RF-063 a RF-067)

ID	Descripción	Prioridad
RF-063	El sistema debe generar notificaciones in-app por eventos: nuevo comentario en post propio, mención, like recibido, misión completada, ranking actualizado.	Alta
RF-064	El sistema debe mostrar un indicador (badge) en la campana de notificaciones con el conteo de no leídas.	Alta
RF-065	El sistema debe permitir marcar notificaciones como leídas individualmente o todas a la vez.	Alta
RF-066	El sistema debe enviar notificaciones push vía FCM cuando el usuario tenga la suscripción activa.	Media
RF-067	El sistema debe enviar email transaccional para eventos críticos: registro, recuperación de contraseña, directo próximo.	Media

3.11 Módulo de Administración (RF-068 a RF-075)

ID	Descripción	Prioridad
RF-068	El sistema debe proporcionar un panel de administración accesible solo a usuarios con rol Administrador.	Alta
RF-069	El sistema debe permitir al Administrador listar y buscar usuarios, y cambiar su rol (Miembro, Autor, Moderador, Administrador).	Alta
RF-070	El sistema debe permitir al Administrador o Moderador ocultar posts o comentarios que incumplan las normas, registrando motivo.	Alta
RF-071	El sistema debe permitir al Administrador suspender temporalmente a un usuario (sin acceso a publicar) por un período configurable.	Alta
RF-072	El sistema debe permitir al Administrador gestionar cursos y recursos de cualquier Autor (editar, eliminar).	Media
RF-073	El sistema debe permitir al Administrador configurar las sesiones en vivo (crear, editar, cancelar).	Media
RF-074	El sistema debe mostrar métricas globales en el panel: usuarios activos diarios, posts/día, recursos descargados, retención D7/D30.	Media
RF-075	El sistema debe mantener un log de auditoría inmutable (append-only) de las acciones administrativas realizadas.	Media

Resumen de cobertura: Esta especificación define 75 requisitos funcionales distribuidos en 11 módulos. La prioridad Alta agrupa 51 requisitos (68%), Media 23 (31%) y Baja 1 (1%). El MVP (fase 1) implementará todos los requisitos Alta; las fases 2 y 3 añadirán los Media; los Baja se diferirán al backlog.

Capítulo 4 — Requisitos No Funcionales

Los requisitos no funcionales describen atributos de calidad del sistema que deben verificarse de manera objetiva, generalmente mediante instrumentación o pruebas de carga. Cada requisito incluye una métrica cuantificable y un método de verificación. A diferencia de los requisitos funcionales, los no funcionales rara vez se validan con un escenario BDD aislado; se validan mediante benchmarks, monitoreo continuo y auditorías periódicas.

4.1 Rendimiento

El sistema debe ofrecer tiempos de respuesta predecibles y rápidos. La paginación y la carga diferida son obligatorias para cualquier listado que pueda exceder 20 elementos. Firestore está optimizado para consultas por índice compuesto, por lo que cualquier consulta nueva debe revisarse contra el plan de índices (Capítulo 10).

ID	Métrica	Objetivo	Verificación
RNF-01	Time to First Byte (TTFB) en Firebase Hosting	< 500 ms (p95)	Lighthouse CI en pipeline
RNF-02	Carga inicial de la home (LCP)	< 2.5 s en 4G	Web Vitals en producción
RNF-03	Latencia de Cloud Functions HTTP	< 300 ms (p95) excluyendo cold start	Cloud Monitoring
RNF-04	Cold start de Cloud Functions	< 2 s en el 95% de invocaciones	Cloud Monitoring
RNF-05	Tiempo de consulta a Firestore (lectura simple)	< 100 ms (p99)	Cloud Monitoring
RNF-06	Paginación de foro (20 posts)	< 800 ms incluyendo render	E2E Cypress con perf hooks
RNF-07	Tiempo de carga de imagen de Storage (avatar 200KB)	< 400 ms	Lighthouse

4.2 Escalabilidad

Firebase escala horizontalmente de forma nativa. Firestore distribuye automáticamente los documentos entre shards y Cloud Functions escala a cero cuando no hay tráfico. Las consideraciones de escalabilidad se centran en: (i) diseño de consultas que no requieran escaneos de colección completos, (ii) uso de subcolecciones en lugar de arrays grandes para evitar documentos que excedan 1 MB, y (iii) uso de Cloud Functions para agregaciones que no pueden resolverse con consultas directas.

ID	Métrica	Objetivo	Verificación
RNF-08	Usuarios concurrentes soportados	10.000 sin degradación	Prueba de carga con k6
RNF-09	Documentos en colección de posts	Hasta 1 millón sin paginación lenta	Índices compuestos + cursor pagination

ID	Métrica	Objetivo	Verificación
RNF-10	Cloud Functions concurrentes	Autoescalado hasta 1000 por región	Firebase Functions config
RNF-11	Tamaño máximo de documento Firestore	< 500 KB (límite Firestore 1 MB)	Validación en Cloud Function pre-write

4.3 Seguridad

La seguridad se implementa en tres capas: (i) Firebase App Check para verificar que las peticiones provienen de la app legítima, (ii) reglas de Firestore como única fuente de verdad para autorización a datos, y (iii) validación de entrada en Cloud Functions para operaciones que no pasan por Firestore directamente. El frontend no implementa lógica de autorización; solo oculta acciones visualmente.

ID	Métrica	Objetivo	Verificación
RNF-12	Cobertura de reglas de seguridad Firestore	100% de colecciones con reglas	Audit script en CI
RNF-13	Rate limiting de Cloud Functions sensibles	Máx 5 llamadas/min por usuario en login y registro	Cloud Functions middleware
RNF-14	Cumplimiento OWASP Top 10	0 vulnerabilidades críticas o altas	Snyk + ZAP scan en CI
RNF-15	Sanitización de contenido HTML en posts	Strip de tags <script>, <iframe>, on* attributes	DOMPurify en frontend y validación en backend
RNF-16	Hashing de contraseñas	Firebase Auth (scrypt con sal única)	Heredado de Firebase Auth
RNF-17	Protección contra CSRF	Token anti-CSRF en formularios	Verificación en Cloud Functions

4.4 Disponibilidad y observabilidad

El SLA de Firebase para Firestore y Cloud Functions es del 99.95% en us-central1. No se implementa redundancia multi-región en v1.0 por costo; se contempla para la fase 3 si la base de usuarios supera 10.000 activos mensuales. La observabilidad se basa en Cloud Logging para logs estructurados, Cloud Monitoring para métricas, y Firebase Analytics para eventos de producto.

ID	Métrica	Objetivo	Verificación
RNF-18	Disponibilidad mensual del sistema	>= 99.9%	Status page + Cloud Monitoring SLO
RNF-19	Tiempo de recuperación ante incidente (RTO)	< 2 horas	Runbook documentado

ID	Métrica	Objetivo	Verificación
RNF-20	Logs estructurados en Cloud Functions	100% con trace ID y user ID	Audit log sample en CI
RNF-21	Alertas proactivas	Latencia p95 > 1s o error rate > 1%	Cloud Monitoring alerting policies

4.5 Usabilidad y accesibilidad

AutoDev debe ser usable por personas con discapacidades visuales, motoras y cognitivas. Se sigue WCAG 2.1 nivel AA. La interfaz responsive abarca desde 375px (móvil) hasta 1920px (desktop) con breakpoints en 768px y 1280px. El contraste mínimo de texto contra fondo es 4.5:1, exceptuando texto decorativo. Todos los elementos interactivos son navegables por teclado con focus visible.

ID	Métrica	Objetivo	Verificación
RNF-22	Cumplimiento WCAG 2.1 AA	100% de páginas	axe-core en CI
RNF-23	Contraste de color	>= 4.5:1 para texto normal	Lighthouse accessibility audit
RNF-24	Navegación por teclado	100% de flujos críticos accesibles	Prueba manual + E2E
RNF-25	Soporte de lectores de pantalla	ARIA labels en todos los elementos interactivos	Prueba con NVDA y VoiceOver
RNF-26	Responsive en 375px, 768px, 1280px, 1920px	Sin scroll horizontal, sin contenido cortado	E2E en Playwright con viewports

4.6 Mantenibilidad y compatibilidad

El código debe ser mantenible por un equipo pequeño (3-5 desarrolladores). Se exige TypeScript estricto, cobertura mínima del 80% en líneas y 70% en ramas para Cloud Functions. El frontend se organiza por features (módulos) con co-localización de componentes, hooks, tipos y pruebas. La compatibilidad aborda las últimas dos versiones de los navegadores mayoritarios; IE11 no se soporta.

ID	Métrica	Objetivo	Verificación
RNF-27	Cobertura de pruebas unitarias (Cloud Functions)	>= 80% líneas, 70% ramas	Jest + Istanbul en CI
RNF-28	Linting sin errores	0 errores ESLint, 0 errores TypeScript	CI gate
RNF-29	Tamaño máximo de archivo	500 líneas (excepto generados)	ESLint rule max-lines
RNF-30	Navegadores soportados	Últimas 2 versiones de Chrome, Firefox, Safari, Edge	Browserlist + Playwright

ID	Métrica	Objetivo	Verificación
RNF-31	i18n preparado	Textos extraídos en /messages/es.json	Audit script en CI

Capítulo 5 — Casos de Uso Detallados

5.1 Diagrama de casos de uso (descripción textual)

Dado que este documento se distribuye en formato PDF y la notación gráfica UML puede comprometerse al rasterizar, se presenta el modelo de casos de uso en forma tabular. La tabla lista cada actor, los casos de uso en los que participa como primario y los casos en los que participa como secundario. El detalle de cada caso de uso se encuentra en las secciones 5.3 a 5.14.

Actor	Casos de uso primarios	Casos de uso secundarios
Invitado	CU-001 Registro de cuenta, CU-004 Login	—
Miembro	CU-002 Onboarding, CU-003 Publicar post, CU-004 Comentar post, CU-005 Filtrar miembros, CU-007 Inscribirse a directo, CU-008 Ver ranking, CU-009 Completar misión, CU-010 Recibir notificación, CU-012 Buscar recurso	CU-011 (mencionado en notificaciones)
Autor	CU-006 Publicar recurso	Todos los del Miembro
Moderador	CU-011 Moderar contenido	—
Administrador	CU-011 Moderar (extendido), gestión usuarios, métricas	—

5.2 Plantilla de caso de uso

Cada caso de uso se documenta siguiendo la plantilla: identificador, nombre, actor primario, precondiciones, postcondiciones, flujo principal numerado, flujos alternativos y excepciones. Esta estructura facilita la conversión directa a escenarios BDD en el Capítulo 7.

5.3 CU-001 — Registro de cuenta

Campo	Valor
ID	CU-001
Nombre	Registro de cuenta
Actor primario	Invitado
Precondiciones	El invitado no tiene sesión activa. El email no está registrado.
Postcondiciones	Se crea el documento de usuario en Firestore con estado "onboarding_pending". Se envía email de verificación. Se redirige al wizard de onboarding.

Flujo principal:

- 1 El invitado accede a la página /registro.
- 2 El sistema muestra el formulario con campos: email, contraseña, confirmar contraseña.

- 3 El invitado completa los campos y pulsa "crear_cuenta".
- 4 El frontend valida formato de email y longitud mínima de contraseña (8 caracteres).
- 5 El frontend llama a Firebase Auth: `createUserWithEmailAndPassword(email, password)`.
- 6 Firebase Auth crea el usuario y devuelve el UID.
- 7 Una Cloud Function `onUserCreate` (trigger `auth.user().onCreate`) crea el documento `/users/{uid}` con campos: `email`, `role="member"`, `status="onboarding_pending"`, `createdAt`.
- 8 El sistema envía email de verificación vía Firebase Auth `sendEmailVerification()`.
- 9 El sistema redirige al usuario al wizard de onboarding (paso 1).

Flujos alternativos:

- **A1 — Email ya registrado:** En el paso 5, Firebase Auth devuelve error `auth/email-already-in-use`. El sistema muestra mensaje "Ya existe una cuenta con este email" y ofrece enlace a `/login`.
- **A2 — Contraseña débil:** En el paso 5, Firebase Auth devuelve error `auth/weak-password`. El sistema resalta el campo contraseña con mensaje "Mínimo 8 caracteres".
- **A3 — Email inválido:** El paso 4 rechaza antes de llamar a Firebase. El sistema muestra "Email no válido" junto al campo.

Excepciones:

- **E1 — Firebase Auth no disponible:** El sistema muestra mensaje genérico "Servicio no disponible temporalmente" y registra el error en Cloud Logging con trace ID.
- **E2 — Cloud Function `onUserCreate` falla:** El usuario se crea en Auth pero no en Firestore. Una Cloud Function de reconciliación (scheduled, cada 5 min) detecta huérfanos y los corrige.

5.4 CU-002 — Onboarding de perfil

Campo	Valor
ID	CU-002
Nombre	Onboarding de perfil
Actor primario	Miembro (recién registrado)
Precondiciones	El usuario tiene cuenta autenticada. Su documento en Firestore tiene <code>status="onboarding_pending"</code> .
Postcondiciones	El documento <code>/users/{uid}</code> se actualiza con <code>name</code> , <code>interests[]</code> , <code>level</code> , <code>bio</code> , <code>status="active"</code> . Se otorga XP inicial (50 XP). Se publica un post automático de presentación en el foro.

Flujo principal:

- 1 El usuario es redirigido a `/onboarding/paso-1` tras el registro.
- 2 Paso 1: Captura del nombre (mínimo 2 caracteres). El usuario pulsa "continuar".

- 3 Paso 2: Selección de al menos 1 interés entre Automatización, IA, Desarrollo Web, Comunidad. Botón "continuar" habilitado solo cuando hay selección.
- 4 Paso 3: Selección de exactamente 1 nivel (Principiante, Intermedio, Avanzado).
- 5 Paso 4: Captura de presentación personal (mínimo 20 caracteres). El usuario pulsa "guardar_perfil && continuar".
- 6 El frontend valida todos los campos antes de enviar.
- 7 El frontend llama a la Cloud Function HTTP completeOnboarding con los datos consolidados.
- 8 La Cloud Function valida los datos, actualiza el documento /users/{uid} (status="active"), otorga 50 XP y crea un post automático en /posts con tag "presentacion".
- 9 El sistema redirige al usuario al /foro.

Flujos alternativos:

- **A1 — Saltar paso opcional:** Si el usuario intenta cerrar la pestaña, el sistema mantiene el status="onboarding_pending" y vuelve a mostrar el wizard en el siguiente login.
- **A2 — Volver atrás:** El usuario puede navegar entre pasos con los botones "atrás" sin perder datos en memoria.

5.5 CU-003 — Publicar post en foro

Campo	Valor
ID	CU-003
Nombre	Publicar post en foro
Actor primario	Miembro
Precondiciones	Usuario autenticado con status="active".
Postcondiciones	Se crea un documento en /posts con autor, contenido, etiquetas y timestamp. Se otorgan 25 XP al autor. Se actualiza el contador postsCount del autor.

Flujo principal:

- 1 El Miembro accede a /foro y localiza el input "¿qué estás construyendo hoy?".
- 2 El Miembro pulsa el input y se expande el formulario con título, contenido y selector de etiquetas.
- 3 El Miembro completa título (máx 100), contenido (máx 5000) y selecciona hasta 3 etiquetas.
- 4 El Miembro pulsa "publicar".
- 5 El frontend valida longitudes y formato, sanitiza HTML con DOMPurify.
- 6 El frontend escribe directamente en /posts mediante set() con uid autogenerado.
- 7 La Cloud Function onPostCreate (trigger firestore onCreate) valida reglas, otorga 25 XP al autor e incrementa postsCount.

8 El post aparece en la cabecera del listado del foro.

Flujos alternativos:

- **A1 — Contenido vacío:** El paso 5 rechaza y muestra "El contenido no puede estar vacío".
- **A2 — Palabra prohibida:** El paso 5 detecta una palabra de la lista negra y solicita revisión.
- **A3 — Rate limit:** Si el usuario publica más de 3 posts en 10 minutos, el sistema bloquea con mensaje "Espera unos minutos antes de publicar de nuevo".

5.6 CU-004 — Comentar post

Campo	Valor
ID	CU-004
Nombre	Comentar en un post
Actor primario	Miembro
Precondiciones	Usuario autenticado con status="active". Existe el post objetivo.
Postcondiciones	Se crea un documento en /posts/{postId}/comments/{commentId}. Se otorgan 10 XP al comentarador. Se notifica al autor del post.

Flujo principal:

- 1 El Miembro abre la vista de detalle de un post.
- 2 El Miembro escribe su comentario en el área inferior y pulsa "comentar".
- 3 El frontend valida longitud (1-1000) y sanitiza.
- 4 El frontend escribe en /posts/{postId}/comments con add().
- 5 La Cloud Function onCommentCreate otorga 10 XP al comentarador, incrementa commentsCount del post y genera una notificación in-app para el autor del post.
- 6 El comentario aparece en la lista con animación.

5.7 CU-005 — Filtrar miembros por especialidad

Campo	Valor
ID	CU-005
Nombre	Filtrar miembros por especialidad
Actor primario	Miembro
Precondiciones	Usuario autenticado.

Campo	Valor
Postcondiciones	El sistema muestra la lista de miembros que cumplen el filtro seleccionado, ordenados según criterio.

Flujo principal:

- 1 El Miembro accede a /miembros.
- 2 El sistema muestra la lista de miembros por defecto (recientes, todos).
- 3 El Miembro pulsa el filtro "Automatización" en la barra de categorías.
- 4 El frontend ejecuta una consulta Firestore: `where("interests", "array-contains", "automatizacion") orderBy("createdAt", "desc")`.
- 5 El sistema actualiza la lista con los miembros filtrados.
- 6 El Miembro puede combinar el filtro de especialidad con el de nivel y el orden.

5.8 CU-006 — Publicar recurso

Campo	Valor
ID	CU-006
Nombre	Publicar recurso
Actor primario	Autor
Precondiciones	Usuario con rol "autor" o "admin".
Postcondiciones	Se crea un documento en /resources con tipo, nivel, imagen destacada y contenido. Aparece en el listado público.

Flujo principal:

- 1 El Autor pulsa "+ publicar" en /recursos.
- 2 El sistema muestra el formulario: título, descripción, tipo (Skill, Plugin, Subagent, MCP Server, Agent Team, Tutorial), nivel (Principiante, Intermedio, Avanzado), imagen destacada, contenido o URL externa.
- 3 El Autor completa los campos y sube una imagen (JPG/PNG/WebP máx 2MB).
- 4 El frontend sube la imagen a Storage en `/resources/{resourceId}/cover.jpg` y obtiene la URL firmada.
- 5 El frontend escribe el documento en /resources con `coverUrl`, `authorId`, `createdAt`.
- 6 El recurso aparece en la lista pública.

5.9 CU-007 — Inscribirse a directo

Campo	Valor
ID	CU-007
Nombre	Reservar cupo en directo
Actor primario	Miembro
Precondiciones	Existe un directo programado con cupos disponibles.
Postcondiciones	Se crea un documento en /lives/{liveId}/attendees/{uid}. Se programa notificación recordatorio 1h antes.

5.10 CU-008 — Visualizar ranking semanal

Campo	Valor
ID	CU-008
Nombre	Visualizar ranking semanal
Actor primario	Miembro
Precondiciones	Existe al menos un período semanal con datos.
Postcondiciones	El sistema muestra el leaderboard con top 3 destacado y tabla del top 100. Muestra la posición del usuario actual si está en el top.

5.11 CU-009 — Completar misión

Campo	Valor
ID	CU-009
Nombre	Completar misión
Actor primario	Miembro
Precondiciones	El usuario tiene una misión activa con tareas pendientes.
Postcondiciones	La misión se marca como completada. Se otorga XP bonus (default 100 XP). Se genera logro si aplica. Se envía notificación.

5.12 CU-010 — Recibir notificación push

Campo	Valor
ID	CU-010
Nombre	Recibir notificación push

Campo	Valor
Actor primario	Miembro (con suscripción push activa)
Precondiciones	El usuario ha aceptado notificaciones push en su navegador/dispositivo.
Postcondiciones	El dispositivo recibe la notificación push. Se registra impresión en /notifications/{uid}/items/{notifId}.

5.13 CU-011 — Moderar contenido (admin/mod)

Campo	Valor
ID	CU-011
Nombre	Moderar contenido inapropiado
Actor primario	Moderador o Administrador
Precondiciones	Existe un post o comentario reportado o detectado.
Postcondiciones	El contenido se marca como hidden=true. Se registra entrada en /audit_logs con motivo, moderatorId y timestamp. Se notifica al autor.

5.14 CU-012 — Buscar recurso por tipo

Campo	Valor
ID	CU-012
Nombre	Buscar recurso por tipo
Actor primario	Miembro
Precondiciones	Existen recursos publicados.
Postcondiciones	El sistema muestra los recursos del tipo seleccionado, ordenados por recientes o popularidad.

Capítulo 6 — Criterios de Aceptación

Los criterios de aceptación (CA-NNN) traducen cada requisito funcional a una condición verificable. Un criterio es aceptable cuando puede responder de forma binaria (sí/no) a la pregunta "¿el sistema cumple este comportamiento bajo estas condiciones?". Cada criterio se asocia a un requisito funcional y a un método de verificación: manual (prueba exploratoria), E2E (Cypress/Playwright) o unit (Jest sobre Cloud Functions).

6.1 Criterios de aceptación por módulo

ID	RF	Criterio	Verificación
CA-001	RF-001	Dado un invitado en /registro, cuando completa email válido y contraseña de 8+ caracteres y pulsa crear_cuenta, entonces se crea el usuario y se redirige a /onboarding.	E2E
CA-002	RF-002	Dado un email ya registrado, cuando el invitado intenta registrarse, entonces el sistema muestra "Ya existe una cuenta con este email".	E2E
CA-003	RF-004	Dado credenciales válidas, cuando el usuario pulsa iniciar_sesión, entonces se establece la sesión y se redirige a /foro.	E2E
CA-004	RF-006	Dado un email registrado, cuando el usuario solicita recuperar contraseña, entonces se envía un email con enlace de reseteo.	Manual
CA-005	RF-007	Dado 5 intentos fallidos consecutivos, cuando el usuario intenta un 6º login, entonces el sistema bloquea por 15 minutos.	Unit
CA-006	RF-009	Dado un usuario recién registrado, cuando accede a cualquier ruta protegida, entonces es redirigido a /onboarding.	E2E
CA-007	RF-010	Dado el paso 1 del onboarding, cuando el usuario ingresa nombre de 1 carácter, entonces el sistema impide continuar.	Unit
CA-008	RF-011	Dado el paso 2 del onboarding, cuando el usuario no selecciona ningún interés, entonces el botón continuar está deshabilitado.	E2E
CA-009	RF-013	Dado el paso 4 del onboarding, cuando el usuario ingresa presentación de menos de 20 caracteres, entonces se muestra error.	E2E
CA-010	RF-014	Dado un usuario con status=onboarding_pending, cuando intenta acceder a /foro, entonces es redirigido a /onboarding.	E2E
CA-011	RF-019	Dado un Miembro activo, cuando publica un post con título y contenido válidos, entonces el post aparece en /foro.	E2E
CA-012	RF-020	Dado un post creado hace menos de 30 min, cuando el autor pulsa editar, entonces puede modificar el contenido.	E2E
CA-013	RF-021	Dado un post del usuario, cuando el autor pulsa eliminar, entonces el post desaparece del listado.	E2E
CA-014	RF-022	Dado un post, cuando un Miembro comenta, entonces el comentario aparece bajo el post.	E2E
CA-015	RF-023	Dado un post, cuando un Miembro pulsa like, entonces el contador incrementa y el botón cambia de estado.	E2E

ID	RF	Criterio	Verificación
CA-016	RF-02 4	Dado el foro con más de 20 posts, cuando el usuario navega a la página 2, entonces se muestran los posts 21-40 sin recargar.	E2E
CA-017	RF-02 5	Dado el foro, cuando el usuario filtra por etiqueta IA, entonces solo se muestran posts con esa etiqueta.	E2E
CA-018	RF-02 6	Dado cualquier post en el listado, entonces se muestra avatar, nombre, nivel, fecha relativa y conteos.	Manual
CA-019	RF-02 9	Dado el catálogo de cursos, cuando el usuario accede, entonces se listan todos los cursos con metadatos.	E2E
CA-020	RF-03 1	Dado un usuario inscrito en un curso, cuando marca una lección como completada, entonces el progreso del curso se actualiza.	E2E
CA-021	RF-03 6	Dado un Autor, cuando completa el formulario de recurso y sube imagen, entonces el recurso se publica.	E2E
CA-022	RF-03 8	Dado el listado de recursos, cuando el usuario filtra por tipo MCP Server, entonces solo se muestran recursos de ese tipo.	E2E
CA-023	RF-04 4	Dado el calendario de directos, cuando el usuario accede, entonces ve los directos programados con cupo.	E2E
CA-024	RF-04 5	Dado un directo con cupos, cuando un Miembro reserva, entonces se confirma y aparece en su lista.	E2E
CA-025	RF-04 6	Dado un directo sin cupos, cuando un Miembro intenta reservar, entonces se le ofrece lista de espera.	E2E
CA-026	RF-04 9	Dado el directorio de miembros, cuando el usuario accede, entonces se muestran los miembros con sus metadatos.	E2E
CA-027	RF-05 0	Dado el directorio, cuando el usuario filtra por nivel Avanzado, entonces solo se muestran miembros avanzados.	E2E
CA-028	RF-05 4	Dado un período semanal cerrado, cuando el sistema calcula el leaderboard, entonces se muestran los 100 miembros con más XP.	Unit
CA-029	RF-05 5	Dado el leaderboard, cuando se visualiza, entonces el top 3 se muestra con medallas oro/plata/bronce.	Manual
CA-030	RF-05 6	Dado lunes 00:00 UTC-4, cuando se ejecuta la Cloud Function scheduled, entonces el leaderboard se reinicia y se archiva el anterior.	Unit
CA-031	RF-05 8	Dado un Miembro que publica un post, cuando la Cloud Function onPostCreate se ejecuta, entonces se otorgan 25 XP.	Unit
CA-032	RF-05 9	Dado un usuario con avatar + presentación + 1 interés, cuando se completa el perfil, entonces la misión "Completa tu perfil" se marca como completada.	Unit
CA-033	RF-06 3	Dado un post con nuevo comentario, cuando la Cloud Function onCommentCreate se ejecuta, entonces se genera una notificación in-app para el autor.	Unit
CA-034	RF-06 4	Dado notificaciones no leídas, cuando el usuario abre la app, entonces la campana muestra un badge con el conteo.	E2E

ID	RF	Criterio	Verificación
CA-035	RF-06 5	Dado notificaciones no leídas, cuando el usuario pulsa "marcar todas como leídas", entonces el badge desaparece.	E2E
CA-036	RF-06 6	Dado un usuario con suscripción push activa, cuando ocurre un evento notificable, entonces recibe push vía FCM.	Manual
CA-037	RF-06 8	Dado un Miembro sin rol admin, cuando intenta acceder a /admin, entonces es redirigido a /foro.	E2E
CA-038	RF-06 9	Dado un Admin, cuando cambia el rol de un usuario a Moderador, entonces el usuario afectado gana permisos de moderación.	E2E
CA-039	RF-07 0	Dado un Moderador, cuando oculta un post, entonces el post desaparece del foro público y se registra en audit_logs.	E2E
CA-040	RF-07 1	Dado un Admin, cuando suspende a un usuario por 7 días, entonces el usuario no puede publicar durante ese período.	E2E

Cobertura: Se definen 40 criterios de aceptación clave que cubren los 11 módulos. El método de verificación E2E cubre 26 criterios, Unit cubre 9 y Manual cubre 5. La matriz de trazabilidad completa (RF → CU → CA → Escenario BDD) se encuentra en el Apéndice B.

Capítulo 7 — Escenarios de Prueba Automatizables (BDD)

Este capítulo presenta los escenarios BDD escritos en lenguaje Gherkin (Given/When/Then) que automatizan la verificación de los criterios de aceptación definidos en el Capítulo 6. Cada escenario es ejecutable con Cucumber.js sobre Node.js usando el Firebase Emulator Suite como entorno de pruebas. Los tags @smoke, @regression y @critical permiten segmentar la ejecución en el pipeline CI/CD: @smoke corre en cada push, @regression corre en cada pull request y @critical corre antes de cada despliegue a producción.

7.1 Feature: Autenticación

@smoke @critical

Feature: Autenticación de usuarios

Como usuario invitado

Quiero poder registrarme e iniciar sesión

Para participar en la comunidad AutoDev

Scenario: Registro exitoso con email y contraseña válidos

Given un invitado en la página "/registro"
And no existe una cuenta con email "ana@dev.com"
When completa el email "ana@dev.com"
And completa la contraseña "SecurePass123"
And confirma la contraseña "SecurePass123"
And pulsa el botón "crear_cuenta"
Then se crea una cuenta en Firebase Auth con email "ana@dev.com"
And se crea un documento en /users con status "onboarding_pending"
And se envía un email de verificación
And el usuario es redirigido a "/onboarding/paso-1"

Scenario: Registro fallido por email duplicado

Given un invitado en la página "/registro"
And ya existe una cuenta con email "ana@dev.com"
When completa el email "ana@dev.com"
And completa la contraseña "SecurePass123"
And confirma la contraseña "SecurePass123"
And pulsa el botón "crear_cuenta"
Then se muestra el mensaje "Ya existe una cuenta con este email"
And se muestra un enlace a "/login"

Scenario: Registro fallido por contraseña corta

Given un invitado en la página "/registro"
When completa el email "ana@dev.com"
And completa la contraseña "abc"
And pulsa el botón "crear_cuenta"
Then se muestra el mensaje "Mínimo 8 caracteres" junto al campo contraseña
And no se crea ninguna cuenta en Firebase Auth

@regression

Scenario: Login exitoso con credenciales válidas

Given un usuario con email "ana@dev.com" y contraseña "SecurePass123"
When el usuario navega a "/login"
And completa el email "ana@dev.com"
And completa la contraseña "SecurePass123"
And pulsa el botón "iniciar_sesion"
Then se establece la sesión del usuario
And se redirige a "/foro"

Scenario: Bloqueo tras 5 intentos fallidos de login

Given un usuario con email "ana@dev.com" y contraseña "SecurePass123"
When el usuario intenta iniciar sesión 5 veces con contraseña incorrecta "WrongPass"
Then la cuenta se bloquea por 15 minutos
And se muestra el mensaje "Demasiados intentos. Intenta nuevamente en 15 minutos"

7.2 Feature: Onboarding

@smoke

Feature: Onboarding de perfil

Como nuevo miembro

Quiero completar mi perfil guiadamente

Para personalizar mi experiencia y presentarme a la comunidad

Scenario: Onboarding completo en 4 pasos

Given un usuario recién registrado con status "onboarding_pending"

When el usuario es redirigido a "/onboarding/paso-1"

And completa el nombre "Ana Martínez"

And pulsa "continuar"

Then avanza al paso 2

When selecciona los intereses "Automatización" e "IA"

And pulsa "continuar"

Then avanza al paso 3

When selecciona el nivel "Intermedio"

And pulsa "continuar"

Then avanza al paso 4

When escribe la presentación "Soy Ana, llevo una tienda online y quiero automatizar con IA"

And pulsa "guardar_perfil && continuar"

Then se actualiza el documento /users/{uid} con status "active"

And se otorgan 50 XP al usuario

And se publica un post automático en /posts con tag "presentacion"

And el usuario es redirigido a "/foro"

Scenario: Validación de nombre corto en paso 1

Given un usuario en "/onboarding/paso-1"

When completa el nombre "A"

Then el botón "continuar" está deshabilitado

And se muestra "Mínimo 2 caracteres"

Scenario: Validación de presentación corta en paso 4

Given un usuario en "/onboarding/paso-4"

When escribe la presentación "Hola"

And pulsa "guardar_perfil && continuar"

Then se muestra "La presentación debe tener al menos 20 caracteres"

And no se actualiza el documento /users/{uid}

7.3 Feature: Foro

```
@smoke @critical
Feature: Foro comunitario
  Como miembro activo
  Quiero publicar, comentar y reaccionar a posts
  Para compartir conocimiento con la comunidad

  Scenario: Publicar un post válido
    Given un miembro autenticado con status "active"
    When navega a "/foro"
    And pulsa el input "¿qué estás construyendo hoy?"
    And completa el título "Mi primer workflow en n8n"
    And completa el contenido "Hoy automatizar mi inbox con n8n..."
    And selecciona la etiqueta "automatizacion"
    And pulsa "publicar"
    Then se crea un documento en /posts con los datos ingresados
    And se otorgan 25 XP al autor
    And el post aparece en la cabecera del listado

  Scenario: Rechazo de post con contenido vacío
    Given un miembro autenticado en "/foro"
    When pulsa el input de creación
    And completa el título "Mi post"
    And deja el contenido vacío
    And pulsa "publicar"
    Then se muestra "El contenido no puede estar vacío"
    And no se crea ningún documento en /posts

  Scenario: Rate limit al publicar más de 3 posts en 10 minutos
    Given un miembro autenticado que ha publicado 3 posts en los últimos 10 minutos
    When intenta publicar un 4º post
    Then se muestra "Espera unos minutos antes de publicar de nuevo"
    And no se crea el 4º post

  Scenario: Comentar en un post
    Given un miembro autenticado
    And un post existente con ID "post-123"
    When el miembro abre el detalle del post "post-123"
    And escribe el comentario "¡Excelente aporte, gracias!"
    And pulsa "comentar"
    Then se crea un documento en /posts/post-123/comments
    And se otorgan 10 XP al comentador
    And se genera una notificación in-app para el autor del post

  Scenario: Dar like a un post (toggle)
    Given un miembro autenticado
    And un post existente con ID "post-123" sin like del miembro
    When el miembro pulsa el botón like del post "post-123"
    Then el contador de likes del post incrementa en 1
    And el botón cambia a estado "liked"
    When el miembro pulsa nuevamente el botón like
    Then el contador de likes decrementa en 1
    And el botón cambia a estado "not-liked"

  Scenario: Filtrar foro por etiqueta IA
    Given un miembro en "/foro"
    And existen posts con etiquetas "automatizacion", "ia", "webapps"
    When el miembro pulsa el filtro "ia"
    Then solo se muestran los posts con etiqueta "ia"
    And se actualiza la URL a "/foro?tag=ia"
```

7.4 Feature: Recursos

```
@regression
Feature: Recursos comunitarios
  Como autor
  Quiero publicar recursos técnicos
  Para que otros miembros puedan aprender y reutilizarlos

  Scenario: Publicar un recurso tipo Skill
    Given un usuario autenticado con rol "autor"
    When navega a "/recursos"
    And pulsa "+ publicar"
    And completa el título "Claude Code Skill: PDF Extractor"
    And completa la descripción "Skill para extraer texto de PDFs"
    And selecciona el tipo "Skill"
    And selecciona el nivel "Principiante"
    And sube una imagen "cover.jpg" de 200KB
    And completa el contenido markdown
    And pulsa "publicar"
    Then se sube la imagen a Storage en /resources/{resourceId}/cover.jpg
    And se crea el documento en /resources con coverUrl, authorId, createdAt
    And el recurso aparece en el listado público

  Scenario: Filtrar recursos por tipo MCP Server
    Given un miembro en "/recursos"
    And existen recursos de tipos "Skill", "Plugin", "MCP Server"
    When el miembro pulsa el filtro "MCP Server"
    Then solo se muestran los recursos de tipo "MCP Server"
```

7.5 Feature: Directos

```
@regression
Feature: Directos en vivo
  Como miembro
  Quiero reservar cupo en directos
  Para participar en sesiones de aprendizaje en tiempo real

  Scenario: Reservar cupo en directo con disponibilidad
    Given un miembro autenticado
    And un directo programado con ID "live-001" y 50 cupos totales
    And 30 cupos ya reservados
    When el miembro pulsa "reservar cupo" en el directo "live-001"
    Then se crea un documento en /lives/live-001/attendees/{uid}
    And el contador de cupos disponibles muestra 19
    And se programa una notificación recordatorio 1 hora antes

  Scenario: Lista de espera cuando el cupo está agotado
    Given un miembro autenticado
    And un directo "live-001" con 50/50 cupos reservados
    When el miembro pulsa "reservar cupo"
    Then se muestra "Cupo agotado"
    And se ofrece la opción "Unirse a lista de espera"
    When el miembro pulsa "Unirse a lista de espera"
    Then se crea un documento en /lives/live-001/waitlist/{uid}
```

7.6 Feature: Ranking y Gamificación

@critical

Feature: Ranking semanal

Como miembro

Quiero ver mi posición en el ranking semanal

Para motivarme a participar más activamente

Scenario: Cálculo automático del leaderboard semanal

Given un período semanal del 06 al 12 de Julio 2026

And usuarios con XP semanal acumulada:

uid xp
u-001 850
u-002 720
u-003 615
u-004 500

When se ejecuta la Cloud Function scheduled computeWeeklyRanking

Then se crea un documento en /rankings/2026-W28 con el top 100

And el top 3 del leaderboard es u-001 (oro), u-002 (plata), u-003 (bronce)

Scenario: Reseteo semanal y premiación

Given es lunes 13 de Julio 2026 a las 00:00 UTC-4

And el ranking de la semana 2026-W28 está cerrado

When se ejecuta la Cloud Function scheduled resetWeeklyRanking

Then el ranking 2026-W28 se archiva en /rankings/2026-W28

And se otorgan 500 XP bonus a u-001

And se otorgan 300 XP bonus a u-002

And se otorgan 200 XP bonus a u-003

And el ranking actual 2026-W29 se inicializa vacío

Scenario: Completar misión tras 3 comentarios

Given un miembro con misión activa "Comenta en 3 posts" con 2 comentarios previos

When el miembro comenta en un 3º post

Then la misión "Comenta en 3 posts" se marca como completada

And se otorgan 100 XP bonus al miembro

And se genera una notificación in-app "¡Misión completada!"

And si el miembro no tenía el logro "Primeros 3 comentarios", se desbloquea

7.7 Feature: Notificaciones

@regression

Feature: Notificaciones in-app y push

Como miembro

Quiero recibir notificaciones de actividad relevante

Para estar al tanto de interacciones con mi contenido

Scenario: Notificación al recibir comentario

Given un post de autor "u-001" con ID "post-123"

And un miembro "u-002" distinto del autor

When "u-002" comenta en "post-123"

Then se crea una notificación en /notifications/u-001/items

And la notificación tiene tipo "new_comment"

And la notificación tiene data.postId="post-123"

And la notificación tiene data.fromUserId="u-002"

And el badge de notificaciones no leídas de "u-001" incrementa en 1

Scenario: Marcar todas las notificaciones como leídas

Given un miembro "u-001" con 5 notificaciones no leídas

When "u-001" pulsa "marcar todas como leídas"

Then todas las notificaciones en /notifications/u-001/items se actualizan con read=true

And el badge de notificaciones no leídas muestra 0

7.8 Feature: Administración

@critical

Feature: Moderación de contenido

Como moderador

Quiero ocultar contenido inapropiado

Para mantener la calidad de la comunidad

Scenario: Ocultar un post reportado

Given un moderador autenticado

And un post "post-999" reportado por 3 usuarios

When el moderador abre el panel de moderación

And selecciona el post "post-999"

And pulsa "ocultar" con motivo "Spam"

Then el documento /posts/post-999 se actualiza con hidden=true

And se crea una entrada en /audit_logs con moderatorId, postId, motivo="Spam"

And se notifica al autor del post "Tu post fue oculto por: Spam"

Scenario: Suspende temporalmente a un usuario

Given un administrador autenticado

And un miembro "u-500" con publicaciones recientes de baja calidad

When el administrador abre el perfil de "u-500"

And pulsa "suspender" y selecciona 7 días

Then el documento /users/u-500 se actualiza con suspendedUntil=now+7d

And se registra en /audit_logs

And "u-500" no puede publicar nuevos posts ni comentarios durante 7 días

And se notifica a "u-500" por email

Scenario: Acceso denegado a /admin para Miembro

Given un miembro autenticado con rol "member"

When intenta navegar a "/admin"

Then es redirigido a "/foro"

And se registra un evento de seguridad en Cloud Logging

Capítulo 8 — Estrategia de Pruebas

La estrategia de pruebas de AutoDev sigue el modelo de pirámide invertida: una base amplia de pruebas unitarias, una capa intermedia de pruebas de integración y una capa superior reducida de pruebas end-to-end. Esta distribución maximiza la velocidad de ejecución del pipeline CI/CD mientras mantiene la confianza en el sistema. La filosofía general es: si un bug puede detectarse con una prueba unitaria, no debe depender de una prueba E2E para ser encontrado.

8.1 Pirámide de pruebas

Nivel	Proporción	Stack	Cobertura objetivo	Frecuencia CI
Unitarias	70%	Jest 29 + ts-jest	80% líneas, 70% ramas	Cada push
Integración	20%	Jest + Firebase Emulator Suite	70% líneas	Cada PR
E2E	10%	Cypress 13 + Cucumber	Flujos críticos al 100%	Cada PR + nightly

8.2 Stack de pruebas detallado

El stack de pruebas se selecciona para maximizar la productividad del equipo y la integración con el ecosistema Firebase. Jest cubre unitarias e integración por su velocidad, ecosistema maduro y compatibilidad nativa con TypeScript. Cypress se elige sobre Playwright para E2E porque su sintaxis declarativa y su panel de debug visual reducen la curva de aprendizaje; sin embargo, se mantiene Playwright como backup para pruebas cross-browser críticas. Cucumber.js ejecuta los escenarios Gherkin del Capítulo 7 sobre el Firebase Emulator Suite.

Herramienta	Uso	Versión	Justificación
Jest	Unit + Integration	29.x	Estándar de facto, ecosistema maduro, fast watch mode.
ts-jest	Transpilación TS	29.x	Compatibilidad directa con TypeScript estricto.
Firebase Emulator Suite	Entorno local	latest	Permite ejecutar Auth, Firestore, Functions y Storage localmente sin costo.
Cypress	E2E	13.x	Debug visual, time-travel, grabación de video en fallos.
Cucumber.js	BDD runner	9.x	Ejecuta escenarios Gherkin del Capítulo 7.
Playwright	Cross-browser E2E	1.40+	Backup para pruebas en Safari y Edge que Cypress no cubre nativamente.
k6	Pruebas de carga	latest	Carga realista contra Cloud Functions y Firestore.
Snyk	Análisis de seguridad	latest	Detección de vulnerabilidades en dependencias.
Lighthouse CI	Performance web	latest	Auditoría de Web Vitals en cada deploy.

8.3 Cobertura mínima

La cobertura mínima exigida es del 80% de líneas y 70% de ramas para Cloud Functions, donde la lógica de negocio se centraliza. Para el frontend, se exige 60% de líneas en utilidades y hooks críticos; los componentes presentacionales no requieren cobertura unitaria obligatoria siempre que estén cubiertos por E2E. La cobertura se mide con Istanbul (nyc) integrado en Jest. Un PR que reduzca la cobertura por debajo del umbral bloquea automáticamente el merge.

8.4 CI/CD Pipeline

El pipeline se implementa en GitHub Actions con los siguientes stages: (1) lint y typecheck en paralelo, (2) pruebas unitarias con Jest, (3) build de frontend y functions, (4) pruebas de integración con Emulator Suite, (5) pruebas E2E con Cypress, (6) análisis de seguridad con Snyk, (7) deploy a ambiente staging automático, (8) smoke tests contra staging, (9) deploy a producción manual con aprobación. El pipeline completo tarda aproximadamente 12 minutos en verde.

8.5 Datos de prueba y seed

El seed del Firebase Emulator Suite genera: 50 usuarios con roles variados, 200 posts distribuidos en 4 etiquetas, 500 comentarios, 30 recursos, 5 cursos con 20 lecciones totales, 3 directos programados, 1 ranking semanal con datos históricos. El seed es idempotente y se ejecuta al iniciar el emulador mediante `firebase emulators:exec "node scripts/seed.js"`. Los datos de prueba usan emails `@example.com` para evitar colisiones con usuarios reales.

8.6 Gestión de ambientes

Ambiente	Propósito	Datos	Deploy
Local (Emulator)	Desarrollo y debugging	Seed sintético idempotente	Manual con firebase emulators:start
Staging	QA y validación pre-producción	Seed sintético + datos anonimizados	Automático en cada merge a main
Production	Usuarios finales	Datos reales	Manual con aprobación tras smoke tests

Capítulo 9 — Arquitectura del Sistema

9.1 Vista general

La arquitectura de AutoDev es 100% serverless sobre Firebase. No existen servidores aprovisionados ni contenedores gestionados manualmente. El frontend (Next.js 16) se sirve desde Firebase Hosting con CDN global. La autenticación delega a Firebase Auth. Los datos transaccionales viven en Firestore (NoSQL orientado a documentos). La lógica de negocio sensible, los triggers reactivos y las tareas programadas se ejecutan en Cloud Functions v2. Los archivos binarios (avatares, imágenes de recursos) se alojan en Cloud Storage for Firebase. Las notificaciones push se entregan vía FCM. La analítica de producto se captura con Firebase Analytics y se visualiza en Google Analytics 4.

Esta elección arquitectónica permite escalar a 10.000 usuarios concurrentes sin re-arquitectura, elimina el costo de servidores ociosos (pay-per-use) y descarga al equipo de tareas operativas como parcheo de SO o gestión de certificados TLS. El compromiso es el lock-in a Google Cloud, mitigado por el patrón Repository en el frontend que aísla las llamadas a Firebase detrás de una interfaz intercambiable.

9.2 Componentes y responsabilidades

Componente	Tecnología	Responsabilidad
Frontend Web	Next.js 16 + React 19 + Tailwind CSS 4 + shadcn/ui	Renderizado de UI, manejo de estado cliente, validación de formularios, llamadas a Firebase SDK.
Firebase Hosting	Firebase Hosting + CDN global	Servir assets estáticos y SSR de Next.js, certificados TLS automáticos.
Firebase Auth	Email/password + Email verification + Reset	Registro, login, sesión JWT, verificación de email, recuperación de contraseña.
Firestore	NoSQL orientado a documentos, multi-región	Almacenamiento de usuarios, posts, comentarios, recursos, cursos, directos, notificaciones, audit_logs.
Cloud Functions v2	TypeScript, Cloud Functions SDK v2	Lógica de servidor: triggers de Firestore (XP, notificaciones), HTTP endpoints, scheduled (ranking semanal).
Cloud Storage	Cloud Storage for Firebase	Avatares, imágenes destacadas de recursos, adjuntos de cursos.
Cloud Messaging	Firebase Cloud Messaging (FCM)	Notificaciones push a usuarios suscritos.
Email Transaccional	SendGrid (vía Extension de Firebase)	Emails de verificación, reseteo, recordatorios de directos.
App Check	Firebase App Check con reCAPTCHA Enterprise	Protección contra abuso: solo la app legítima puede llamar a Firestore y Functions.
Analytics	Firebase Analytics + GA4	Captura de eventos de producto: post_created, lesson_completed, etc.
Cloud Logging	Google Cloud Logging	Logs estructurados de Cloud Functions con trace ID y user ID.
Cloud Monitoring	Google Cloud Monitoring	Métricas, SLOs y alertas (latencia, error rate, cold starts).

9.3 Patrones arquitectónicos

9.3.1 Repository Pattern en el frontend

Todas las llamadas a Firebase desde el frontend se encapsulan detrás de interfaces Repository. Por ejemplo, PostRepository expone métodos como create(), list(), comment() y like() que internamente usan el SDK de Firestore. Este patrón facilita mockear Firebase en pruebas unitarias del frontend y permite, en el futuro, migrar a otro backend sin reescribir componentes. Los repositorios se inyectan vía React Context.

9.3.2 Event-driven backend con triggers de Firestore

La lógica reactiva se implementa mediante Cloud Functions reactivas a eventos de Firestore. Cuando un documento se crea o actualiza, Firestore emite un evento que dispara la función. Por ejemplo, onPostCreate se dispara al crear un documento en /posts, valida el contenido, otorga XP al autor y actualiza contadores. Este patrón desacopla la escritura de datos (frontend directo a Firestore) del enriquecimiento y validación de negocio (Cloud Functions).

9.3.3 Serverless con escalado a cero

Cloud Functions v2 escala automáticamente de 0 a N instancias según la carga. El cold start es el principal desafío; se mitiga configurando minInstances=1 en funciones críticas (auth, onboarding) y usando funciones más ligeras (sin dependencias pesadas) para triggers frecuentes. Las funciones HTTP se exponen detrás de Firebase Hosting para aprovechar su CDN y reducir latencia.

9.3.4 Single source of truth en Firestore Rules

Las reglas de seguridad de Firestore son la única fuente de verdad para autorización. El frontend puede ocultar botones o rutas según el rol del usuario, pero esa lógica es cosmética; el backend (Firestore Rules) rechaza cualquier acceso no autorizado. Esta separación evita vulnerabilidades donde un usuario malicioso podría eludir la UI para acceder a datos.

9.4 Flujos de datos principales

9.4.1 Flujo: Publicación de un post

El Miembro completa el formulario en el frontend. El frontend valida y sanitiza localmente, luego escribe directamente en /posts con set() generando un ID automáticamente. La escritura pasa por las reglas de Firestore que validan: (i) el usuario está autenticado, (ii) tiene status="active", (iii) los campos obligatorios están presentes y cumplen longitud máxima. Si las reglas pasan, el documento se persiste. Inmediatamente, el trigger onPostCreate se dispara en Cloud Functions: la función valida nuevamente (defensa en profundidad), otorga 25 XP al autor (update en /users/{uid}), incrementa postsCount del autor y publica un evento en Analytics.

9.4.2 Flujo: Cálculo del ranking semanal

Una Cloud Function scheduled se ejecuta cada lunes a las 00:00 UTC-4. La función consulta /users ordenado por weeklyXP descendientemente, limit 100. Escribe el resultado en /rankings/{year-Wxx} como snapshot inmutable. Otorga XP bonus a los top 3 (500, 300, 200). Resetea el campo weeklyXP de todos los usuarios a 0. Genera notificaciones in-app para los top 100 anunciando su posición.

9.4.3 Flujo: Notificación push tras comentario

El Miembro "u-002" comenta en un post de "u-001". El trigger `onCommentCreate` se dispara: la función crea un documento en `/notifications/u-001/items` con `tipo=new_comment`, `postId`, `fromUserId=u-002` y `timestamp`. Adicionalmente, consulta el documento `/users/u-001` para obtener su FCM token; si existe y el usuario tiene push habilitado, envía el mensaje vía FCM admin SDK. Si el envío falla, se registra el error pero la notificación in-app ya está persistida y será visible cuando el usuario abra la app.

9.5 Seguridad perimetral

La seguridad se organiza en capas concéntricas. La capa exterior es App Check: verifica que las llamadas a Firestore y Cloud Functions provengan de la app legítima (no de un script malicioso). La siguiente capa son las reglas de Firestore (detalladas en el Capítulo 10) que validan autenticación, autorización por rol y forma de los datos. La capa interior es la validación en Cloud Functions: incluso si un documento pasa las reglas, las funciones que reaccionan a su creación realizan una segunda validación de negocio (rate limits, integridad referencial, anti-spam). Esta defensa en profundidad sigue el principio de "never trust input".

9.6 Estrategia multi-región y escalado

En v1.0, Firestore se configura en modo nativo multi-región `us-central` (Centro de EE.UU.) que replica datos automáticamente entre regiones y garantiza 99.999% de disponibilidad para lecturas y escrituras. Cloud Functions se despliegan en `us-central1`. Si la base de usuarios crece significativamente en América Latina, se evaluará en la fase 3 una segunda despliegue en `southamerica-east1` (São Paulo) con Firestore cross-region replication para reducir latencia. Esta decisión se documentará en un ADR específico antes de su implementación.

Capítulo 10 — Diseño de Base de Datos (Firestore)

Firestore es una base de datos NoSQL orientada a documentos organizada en colecciones y subcolecciones. A diferencia de SQL, no hay joins ni esquema rígido; la desnormalización controlada es una práctica recomendada para optimizar lecturas. Este capítulo documenta cada colección, su esquema, índices compuestos necesarios, reglas de seguridad y los triggers de Cloud Functions asociados.

10.1 Modelo de datos general

AutoDev organiza los datos en 11 colecciones raíz: users, posts, courses, lessons, resources, lives, rankings, missions, achievements, notifications y audit_logs. Algunas colecciones tienen subcolecciones para datos jerárquicos (ej. /posts/{postId}/comments/{commentId}). La convención de nombrado es: colecciones en camelCase plural (posts, users), IDs de documentos como UID de Firebase Auth para usuarios o IDs auto-generados para el resto, campos en camelCase.

10.2 Colección: users

Almacena el perfil y estado de cada usuario. Se crea automáticamente por la Cloud Function onUserCreate cuando Firebase Auth emite el evento auth.user().onCreate.

```
// /users/{uid}
{
  uid: string, // = document ID (Firebase Auth UID)
  email: string, // email del usuario
  displayName: string, // nombre para mostrar (onboarding)
  avatarUrl: string | null, // URL firmada de Cloud Storage
  interests: string[], // ["automatizacion", "ia", "webapps", "comunidad"]
  level: "principiante" | "intermedio" | "avanzado",
  bio: string, // presentación personal
  role: "member" | "autor" | "moderador" | "admin",
  status: "onboarding_pending" | "active" | "suspended",
  suspendedUntil: Timestamp | null,
  xp: number, // XP total acumulada
  weeklyXP: number, // XP en la semana actual (reset cada lunes)
  level_number: number, // nivel calculado por XP (1, 2, 3, ...)
  postsCount: number, // contador desnormalizado
  commentsCount: number, // contador desnormalizado
  fcmToken: string | null, // token FCM para push
  pushEnabled: boolean, // preferencia de push
  emailNotifications: boolean, // preferencia de email
  createdAt: Timestamp,
  updatedAt: Timestamp,
  lastActiveAt: Timestamp,
}
```

10.3 Colección: posts y subcolección comments

Cada post es un documento en /posts. Los comentarios viven en la subcolección /posts/{postId}/comments para mantenerlos junto al post padre y permitir consultas eficientes. Los likes se almacenan como mapa (map) dentro del documento del post para evitar una subcolección adicional y facilitar el toggle.

```
// /posts/{postId}
{
  postId: string, // = document ID
  authorId: string, // ref a /users/{uid}
  authorName: string, // denormalizado para mostrar en lista
  authorLevel: number, // denormalizado
  title: string, // máx 100
  content: string, // máx 5000, HTML sanitized
  tags: string[], // máx 3
  likesCount: number, // contador
  likes: { [uid: string]: boolean }, // mapa para toggle 0(1)
  commentsCount: number, // contador
  hidden: boolean, // moderación
  hiddenReason: string | null,
  createdAt: Timestamp,
  updatedAt: Timestamp,
}

// /posts/{postId}/comments/{commentId}
{
  commentId: string,
  postId: string, // denormalizado
  authorId: string,
  authorName: string,
  authorLevel: number,
  content: string, // máx 1000
  likesCount: number,
  likes: { [uid: string]: boolean },
  hidden: boolean,
  createdAt: Timestamp,
  updatedAt: Timestamp,
}
```

10.4 Colección: resources

```
// /resources/{resourceId}
{
  resourceId: string,
  authorId: string,
  authorName: string,
  title: string,
  description: string, // máx 500
  type: "Skill" | "Plugin" | "Subagent" | "MCP Server" | "Agent Team" | "Tutorial",
  level: "Principiante" | "Intermedio" | "Avanzado",
  coverUrl: string, // Storage URL firmada
  content: string, // markdown
  externalUrl: string | null, // URL alternativa (repo, marketplace)
  downloadsCount: number, // contador
  favoritesCount: number,
  createdAt: Timestamp,
  updatedAt: Timestamp,
}
```

10.5 Colección: courses y subcolección lessons

```
// /courses/{courseId}
{
  courseId: string,
  authorId: string,
  title: string,
  description: string,
  coverUrl: string,
  durationMinutes: number,
  lessonsCount: number, // denormalizado
  enrolledCount: number, // denormalizado
  createdAt: Timestamp,
}

// /courses/{courseId}/lessons/{lessonId}
{
  lessonId: string,
  courseId: string,
  title: string,
  content: string, // markdown
  order: number, // 1, 2, 3...
  xpReward: number, // default 50
  createdAt: Timestamp,
}

// /courses/{courseId}/progress/{uid} -- progreso por usuario
{
  uid: string,
  courseId: string,
  completedLessons: string[], // lessonIds
  progress: number, // 0-100
  completedAt: Timestamp | null,
}
```

10.6 Colección: lives y subcolecciones

```
// /lives/{liveId}
{
  liveId: string,
  title: string,
  description: string,
  hostId: string,
  hostName: string,
  scheduledAt: Timestamp,
  durationMinutes: number,
  maxAttendees: number,
  attendeesCount: number, // denormalizado
  status: "scheduled" | "live" | "ended" | "cancelled",
  streamUrl: string | null, // se carga al iniciar
  createdAt: Timestamp,
}

// /lives/{liveId}/attendees/{uid}
{
  uid: string,
  liveId: string,
  registeredAt: Timestamp,
  attended: boolean,
}

// /lives/{liveId}/waitlist/{uid}
{
  uid: string,
  joinedAt: Timestamp,
}
```

10.7 Colección: rankings (histórico)

```
// /rankings/{periodId} -- periodId = "2026-W28"
{
  periodId: string,
  startDate: Timestamp,
  endDate: Timestamp,
  computedAt: Timestamp,
  top100: [
    { uid: string, displayName: string, avatarUrl: string, xp: number, rank: number }
  ],
  rewards: [
    { uid: string, rank: number, xpBonus: number }
  ],
}
```

10.8 Colecciones: missions, achievements, notifications, audit_logs

```

// /missions/{missionId} -- definición de misión
{
  missionId: string,
  title: string,
  description: string,
  xpReward: number,
  tasks: [{ id: string, type: string, target: number }],
}

// /users/{uid}/missions/{userMissionId} -- progreso por usuario
{
  missionId: string,
  uid: string,
  progress: { [taskId: string]: number },
  completed: boolean,
  completedAt: Timestamp | null,
}

// /achievements/{achievementId}
{
  achievementId: string,
  title: string,
  description: string,
  iconUrl: string,
  criteria: string, // descripción legible del criterio
}

// /users/{uid}/achievements/{achievementId} -- desbloques
{
  achievementId: string,
  uid: string,
  unlockedAt: Timestamp,
}

// /notifications/{uid}/items/{notifId}
{
  notifId: string,
  uid: string,
  type: "new_comment" | "new_like" | "mention" | "mission_completed" |
        "rank_update" | "directo_reminder",
  data: { [key: string]: any },
  read: boolean,
  createdAt: Timestamp,
}

// /audit_logs/{logId} -- append-only, immutable
{
  logId: string,
  actorId: string, // admin/mod que ejecutó la acción
  action: "hide_post" | "suspend_user" | "change_role" | ...,
  targetType: string, // "post", "user", "resource"
  targetId: string,
  motivo: string,
  metadata: { [key: string]: any },
  timestamp: Timestamp,
}

```

10.9 Índices compuestos necesarios

Firestore requiere índices compuestos para consultas que combinan filtros y orderBy en campos diferentes. Los índices siguientes se configuran en `firebase.indexes.json` y se despliegan con `firebase deploy --only firestore:indexes`.

Colección	Campos indexados	Consulta que lo usa
posts	tags (array) + createdAt (desc)	Listado de foro filtrado por etiqueta
posts	hidden=false + createdAt (desc)	Foro público sin posts ocultos
posts	authorId + createdAt (desc)	Posts de un usuario en su perfil
users	level + createdAt (desc)	Directorio de miembros filtrado por nivel
users	interests (array) + createdAt (desc)	Miembros filtrados por especialidad
users	weeklyXP (desc)	Leaderboard semanal top 100
users	status=active + lastActiveAt (desc)	Miembros activos hoy
resources	type + createdAt (desc)	Recursos filtrados por tipo
resources	level + createdAt (desc)	Recursos filtrados por nivel
lives	status=scheduled + scheduledAt (asc)	Próximos directos
notifications/{uid}/items	read=false + createdAt (desc)	Notificaciones no leídas
audit_logs	timestamp (desc)	Log de auditoría ordenado

10.10 Reglas de seguridad Firestore

Las reglas de seguridad son la única fuente de verdad para autorización. Se despliegan con *firebase deploy --only firestore:rules*. A continuación se muestran las reglas principales simplificadas y divididas en bloques para facilitar su lectura.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Helper: usuario autenticado y activo
    function isAuthActive() {
      return request.auth != null
        && get(/databases/{database}/documents/users/{request.auth.uid})
          .data.status == 'active';
    }

    function isAuthAdmin() {
      return request.auth != null
        && get(/databases/{database}/documents/users/{request.auth.uid})
          .data.role == 'admin';
    }

    function isAuthModerator() {
      return request.auth != null
        && get(/databases/{database}/documents/users/{request.auth.uid})
          .data.role in ['moderador', 'admin'];
    }
  }
}
```

Bloque 2 — Colección users:

```
// users: lectura pública del perfil (sin campos sensibles)
// escritura solo del propio usuario (campos limitados)
match /users/{uid} {
  allow read: if request.auth != null;
  allow create: if false; // solo via Cloud Function onUserCreate
  allow update: if request.auth.uid == uid
    && request.resource.data.role == resource.data.role // no cambiar rol
    && request.resource.data.xp == resource.data.xp; // no cambiar XP
  allow delete: if isAuthAdmin();
}
```

Bloque 3 — Colección posts y subcolección comments:

```
match /posts/{postId} {
  allow read: if resource.data.hidden == false || isAuthModerator();
  allow create: if isAuthActive()
    && request.resource.data.authorId == request.auth.uid
    && request.resource.data.title.size() <= 100
    && request.resource.data.content.size() <= 5000
    && request.resource.data.tags.size() <= 3;
  allow update: if isAuthModerator()
    || (resource.data.authorId == request.auth.uid
      && request.time.toMillis() < resource.data.createdAt.toMillis() + 1800000);
  allow delete: if resource.data.authorId == request.auth.uid || isAuthAdmin();

  match /comments/{commentId} {
    allow read: if resource.data.hidden == false || isAuthModerator();
    allow create: if isAuthActive()
      && request.resource.data.authorId == request.auth.uid
      && request.resource.data.content.size() <= 1000;
    allow update: if isAuthModerator();
    allow delete: if resource.data.authorId == request.auth.uid || isAuthAdmin();
  }
}
```

Bloque 4 — Colecciones resources, notifications, audit_logs:

```
match /resources/{resourceId} {
  allow read: if true;
  allow create: if isAuthActive()
    && get(/databases/{database}/documents/users/{request.auth.uid}).data.role in ['autor', 'admin'];
  allow update, delete: if resource.data.authorId == request.auth.uid || isAuthAdmin();
}

// notifications: solo el propietario puede leer/escribir sus notifs
match /notifications/{uid}/items/{notifId} {
  allow read, update: if request.auth.uid == uid;
  allow create: if false; // solo Cloud Functions con admin SDK
  allow delete: if false;
}

// audit_logs: solo admin lee, solo Cloud Functions escribe
match /audit_logs/{logId} {
  allow read: if isAuthAdmin();
  allow write: if false; // solo admin SDK desde Cloud Functions
}
```

10.11 Triggers de Cloud Functions

Los triggers son Cloud Functions v2 que reaccionan a eventos de Firestore o a programaciones cron. Implementan la lógica reactiva del sistema: gamificación, notificaciones, contadores desnormalizados, ranking semanal y reconciliación.

Trigger	Evento	Acción
onUserCreate	auth.user().onCreate	Crea /users/{uid} con status="onboarding_pending", role="member", xp=0.
onPostCreate	firestore /posts onCreate	Valida contenido, otorga 25 XP al autor, incrementa postsCount, publica evento Analytics.
onPostUpdate	firestore /posts onUpdate	Si hidden cambia a true, notifica al autor.
onCommentCreate	firestore /posts/{postId}/comments onCreate	Otorga 10 XP al comentador, incrementa commentsCount del post, genera notificación in-app para el autor del post.
onLikeToggle	firestore /posts onUpdate (likes field)	Otorga 5 XP al autor del post cuando recibe un like nuevo, genera notificación.
onLessonComplete	firestore /courses/{courseId}/progress onUpdate	Otorga XP (default 50) al usuario cuando marca una lección como completada.
onResourceView	https onRequest	Incrementa downloadsCount/visitas del recurso.
onLiveRegistration	firestore /lives/{liveId}/attendees onCreate	Actualiza attendeesCount, programa notificación recordatorio 1h antes.
computeWeeklyRanking	schedule 0 0 * * * 1 (lunes 00:00 UTC-4)	Calcula top 100, archiva en /rankings/{periodId}, otorga XP bonus a top 3, resetea weeklyXP.
resetDailyActive	schedule 0 0 * * * (diario)	Marca lastActiveAt daily para métricas DAU.
reconcileOrphanUsers	schedule */5 * * * * (cada 5 min)	Detecta usuarios en Auth sin documento en Firestore y los crea.

10.12 Snippet de Cloud Function: onPostCreate

A modo de ejemplo, se muestra la implementación del trigger onPostCreate que otorga XP al autor tras publicar un post. La función usa el SDK v2 de Firebase Functions con tipado TypeScript estricto.

```

import { onDocumentCreated } from "firebase-functions/v2/firestore";
import { getFirestore, FieldValue } from "firebase-admin/firestore";
import { initializeApp } from "firebase-admin/app";

initializeApp();
const db = getFirestore();

export const onPostCreate = onDocumentCreated(
  {
    document: "posts/{postId}",
    region: "us-central1",
    minInstances: 1,
  },
  async (event) => {
    const snapshot = event.data;
    if (!snapshot) return;

    const post = snapshot.data();
    const authorId = post.authorId;

    // 1. Validar (defensa en profundidad)
    if (!post.title || !post.content || post.content.length > 5000) {
      console.warn("Invalid post content, skipping XP");
      return;
    }

    // 2. Otorgar 25 XP al autor e incrementar postsCount
    const userRef = db.collection("users").doc(authorId);
    await userRef.update({
      xp: FieldValue.increment(25),
      weeklyXP: FieldValue.increment(25),
      postsCount: FieldValue.increment(1),
      lastActiveAt: FieldValue.serverTimestamp(),
    });

    // 3. Publicar evento Analytics
    console.log(JSON.stringify({
      type: "post_created",
      authorId,
      postId: snapshot.id,
      xpAwarded: 25,
      traceId: event.id,
    }));

    // 4. Verificar misiones (delegado a otra función)
    await db.collection("tasks").add({
      type: "check_missions",
      uid: authorId,
      event: "post_created",
      createdAt: FieldValue.serverTimestamp(),
    });
  }
);

```

Capítulo 11 — Roadmap y Entregables

El roadmap de AutoDev se organiza en cuatro fases incrementales, cada una con criterios de salida verificables. La filosofía es entregar valor temprano: el MVP (fase 1) ofrece una comunidad funcional mínima que permite validar la propuesta de valor con usuarios reales antes de invertir en funcionalidades avanzadas. Cada fase se planifica con un horizonte de 6 a 10 semanas de desarrollo por un equipo de 3-5 ingenieros.

11.1 Fases del roadmap

Fase	Duración	Módulos incluidos	Criterio de salida
Fase 1 — MVP	8 semanas	Auth, Onboarding, Foro, Miembros	100 usuarios beta activos, NPS > 30, 0 bugs críticos
Fase 2 — Contenido	6 semanas	Recursos, Cursos, Gamificación (misiones, XP)	50 recursos publicados, 5 cursos activos, retención D7 > 40%
Fase 3 — Engagement	6 semanas	Directos, Ranking, Notificaciones push	Leaderboard operativo, 1000 push enviadas, retención D30 > 25%
Fase 4 — Escala	8 semanas	Panel Admin, Analytics avanzado, Stripe	Admin operativo, dashboard métricas, 10 pagos procesados

11.2 Criterios de salida por fase

Cada fase tiene criterios de salida objetivos y medibles. Ninguna fase puede considerarse completada hasta que todos sus criterios estén verificados. Esto evita la deuda técnica acumulada por "lo arreglamos en la siguiente fase".

- **Fase 1 MVP:** 100 usuarios beta registrados con onboarding completo; al menos 1 post y 1 comentario por usuario en promedio; NPS superior a 30; 0 bugs críticos abiertos; Lighthouse Performance > 80; cobertura de pruebas > 70%.
- **Fase 2 Contenido:** 50 recursos publicados por al menos 5 autores distintos; 5 cursos activos con 20 lecciones totales; retención D7 superior al 40%; sistema de XP operativo y verificado por pruebas de carga.
- **Fase 3 Engagement:** Leaderboard semanal operativo con 4 semanas de histórico; 1000 notificaciones push entregadas con tasa de apertura > 20%; retención D30 superior al 25%; al menos 3 directos realizados con 50 asistentes promedio.
- **Fase 4 Escala:** Panel de admin usado semanalmente por al menos 2 administradores; dashboard de métricas con 10 KPIs en tiempo real; 10 pagos procesados con Stripe sin discrepancias; documentación runbooks completa para incidentes.

11.3 Riesgos y mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación
Costo de Firestore escalando con usuarios	Media	Alto	Cotizar plan Blaze con forecast de lecturas/escrituras; cache en frontend para lecturas frecuentes; denormalización para evitar lecturas múltiples.
Abuso de gamificación (XP farming)	Alta	Medio	Rate limiting en Cloud Functions; detección de patrones anómalos (logs en audit_logs); tope diario de XP por acción.
Contenido inapropiado en foro	Alta	Alto	Filtro de palabras prohibidas en pre-write; reporte de usuarios; moderación proactiva; logs de auditoría inmutables.
Cold start de Cloud Functions críticas	Media	Medio	minInstances=1 en auth y onboarding; separar funciones frecuentes (ligeras) de funciones pesadas.
Lock-in a Firebase	Baja	Medio	Patrón Repository en frontend; ADR documentado antes de migrar; mantener un módulo de abstracción de auth.
Indisponibilidad de Firebase	Baja	Alto	Status page monitoring; runbooks para incidentes; comunicación proactiva a usuarios vía Twitter/email.
Adopción insuficiente de usuarios	Media	Alto	Validar MVP con beta privada antes de invertir en fases 2-4; encuestas NPS semanales; canal directo de feedback.

Capítulo 12 — Apéndices

Apendice A — Glosario completo

El glosario recopila los términos técnicos utilizados a lo largo del documento. Cualquier término no listado conserva su significado estándar en la literatura de ingeniería de software y plataformas Firebase.

Término	Definición
App Check	Servicio Firebase que protege backends contra abuso verificando la identidad de la app cliente.
BDD	Behavior Driven Development. Práctica que expresa requisitos como comportamientos en Gherkin.
Cloud Functions	Funciones serverless de Firebase ejecutadas en respuesta a eventos HTTP, Firestore o scheduled.
FCM	Firebase Cloud Messaging. Servicio de notificaciones push.
Firestore	Base de datos NoSQL orientada a documentos, en tiempo real.
Gamificación	Aplicación de mecánicas de juego (XP, niveles, logros) en contextos no lúdicos.
Gherkin	Lenguaje estructurado (Given/When/Then) para describir escenarios de prueba.
Leaderboard	Tabla de clasificación semanal de usuarios ordenada por XP.
MCP Server	Model Context Protocol Server. Recurso tipificado en AutoDev.
Misión	Conjunto de tareas que otorgan XP bonus al completarse.
Onboarding	Proceso guiado de registro y configuración inicial de un nuevo usuario.
Repository Pattern	Patrón de diseño que aísla el acceso a datos detrás de una interfaz.
SDD	Specification Driven Development. Metodología que prioriza la especificación formal de requisitos.
SRS	Software Requirements Specification. Documento que describe requisitos del sistema.
XP	Experience Points. Unidad de puntaje en el sistema de gamificación.

Apendice B — Matriz de trazabilidad

La matriz de trazabilidad vincula cada requisito funcional con su caso de uso, criterio de aceptación y escenario BDD. Esta matriz es la fuente de verdad para auditar la cobertura: cualquier requisito sin trazabilidad completa debe completarse antes del cierre de la fase correspondiente.

RF	CU	CA	Escenario BDD	Módulo
RF-001	CU-001	CA-001	7.1 Registro exitoso	Auth
RF-002	CU-001	CA-002	7.1 Email duplicado	Auth

RF	CU	CA	Escenario BDD	Módulo
RF-004	CU-001	CA-003	7.1 Login exitoso	Auth
RF-007	—	CA-005	7.1 Bloqueo tras 5 intentos	Auth
RF-009	CU-002	CA-006	7.2 Onboarding completo	Onboarding
RF-010	CU-002	CA-007	7.2 Nombre corto	Onboarding
RF-013	CU-002	CA-009	7.2 Presentación corta	Onboarding
RF-014	—	CA-010	7.2 Redirección onboarding	Onboarding
RF-019	CU-003	CA-011	7.3 Publicar post válido	Foro
RF-020	CU-003	CA-012	—	Foro
RF-021	CU-003	CA-013	—	Foro
RF-022	CU-004	CA-014	7.3 Comentar en un post	Foro
RF-023	CU-004	CA-015	7.3 Dar like a un post	Foro
RF-024	—	CA-016	—	Foro
RF-025	—	CA-017	7.3 Filtrar foro por IA	Foro
RF-029	—	CA-019	—	Cursos
RF-031	—	CA-020	—	Cursos
RF-036	CU-006	CA-021	7.4 Publicar Skill	Recursos
RF-038	CU-012	CA-022	7.4 Filtrar MCP Server	Recursos
RF-044	—	CA-023	—	Directos
RF-045	CU-007	CA-024	7.5 Reservar cupo	Directos
RF-046	CU-007	CA-025	7.5 Lista de espera	Directos
RF-049	—	CA-026	—	Miembros

RF	CU	CA	Escenario BDD	Módulo
RF-050	CU-005	CA-027	—	Miembros
RF-054	CU-008	CA-028	7.6 Cálculo leaderboard	Ranking
RF-055	CU-008	CA-029	—	Ranking
RF-056	—	CA-030	7.6 Reseteo semanal	Ranking
RF-058	—	CA-031	—	Gamificación
RF-059	CU-009	CA-032	7.6 Completar misión	Gamificación
RF-063	—	CA-033	7.7 Notificación al recibir comentario	Notificaciones
RF-064	CU-010	CA-034	—	Notificaciones
RF-065	—	CA-035	7.7 Marcar todas como leídas	Notificaciones
RF-066	CU-010	CA-036	—	Notificaciones
RF-068	—	CA-037	7.8 Acceso denegado a /admin	Admin
RF-069	—	CA-038	—	Admin
RF-070	CU-011	CA-039	7.8 Ocultar post reportado	Admin
RF-071	CU-011	CA-040	7.8 Suspender usuario	Admin

Apéndice C — Convenciones de nombrado

Las convenciones de nombrado aseguran consistencia entre el código frontend, las Cloud Functions y los documentos de Firestore. Cualquier desviación debe justificarse en un ADR.

Recurso	Convención	Ejemplo
Colecciones Firestore	camelCase plural	posts, users, auditLogs, notifications
IDs de documentos	UID de Auth para usuarios; auto-ID para el resto	users/{uid}, posts/{postId}
Campos de documentos	camelCase	displayName, createdAt, weeklyXP
Timestamps	Tipo Timestamp de Firestore	createdAt: Timestamp
Booleans	prefijo is/has cuando aplica	isActive, hasAvatar

Recurso	Convención	Ejemplo
Contadores	sufijo Count	postsCount, commentsCount, attendeesCount
Cloud Functions	camelCase, prefijo on para triggers	onPostCreate, computeWeeklyRanking
Rutas frontend	kebab-case	/onboarding/paso-1, /recursos
Constantes TypeScript	UPPER_SNAKE_CASE	MAX_POST_LENGTH, XP_PER_POST
Interfaces TypeScript	PascalCase, sufijo Repository para repos	PostRepository, UserService
Archivos de prueba	*.test.ts (unit) / *.spec.ts (E2E)	onPostCreate.test.ts, login.spec.ts

Apendice D — Documentos relacionados

Esta especificación se complementa con los siguientes artefactos que se mantienen en el repositorio del proyecto. Cada artefacto tiene un responsable y una cadencia de actualización. La versión de cada documento se incrementa cuando se aprueba un cambio que afecta la especificación.

- **README.md** — Setup del proyecto, comandos de desarrollo y estructura de carpetas. Responsable: Tech Lead. Actualización: por cambio.
- **ADR/001-firebase-stack.md** — Architecture Decision Record justificando la elección de Firebase como stack backend. Responsable: Arquitecto. Actualización: por revisión anual.
- **RUNBOOK/incident-response.md** — Procedimiento paso a paso para incidentes de producción. Responsable: SRE. Actualización: tras cada incidente postmortem.
- **firestore.rules** — Reglas de seguridad desplegadas. Responsable: Backend Lead. Actualización: por cambio de permisos.
- **firestore.indexes.json** — Índices compuestos desplegados. Responsable: Backend Lead. Actualización: por nueva consulta frecuente.
- **/functions/src/** — Código TypeScript de Cloud Functions. Responsable: Backend. Actualización: continua.
- **/e2e/features/** — Escenarios Gherkin del Capítulo 7. Responsable: QA. Actualización: por nuevo criterio de aceptación.